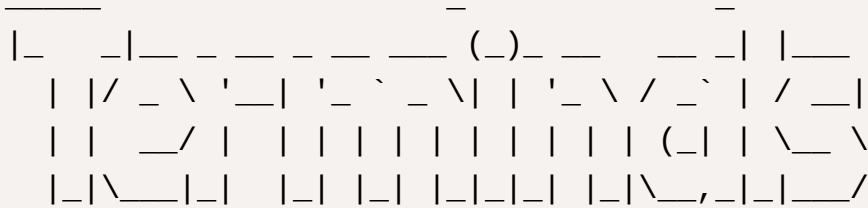


Table of Contents

Terminal Windows: Guia do Desenvolvedor	3
Começando com Terminais Windows	6
Command Prompt (CMD): O Terminal Clássico do Windows	12
CMD: Seu Primeiro Terminal	17
Comandos Básicos: Seu Arsenal Inicial	23
Navegação no CMD: Perdido no Terminal	27
Operações com Arquivos: Dominando o Sistema	31
Comandos do Sistema: Controlando o Windows	39
Técnicas Avançadas de CMD	44
Scripts Batch: Automatize ou Morra Tentando	50
Variáveis de Ambiente: O Coração do Sistema	60
Networking no CMD: Dominando a Rede	65
Automação Avançada: Além dos Scripts	70
Segurança no CMD: Porque Paranoia é Apenas Bom Senso	76
Debugging no CMD: Onde os Erros Vão Para Festa	80
Exercícios CMD: Hora de Praticar	85
Integração CMD: Onde Todos os Mundos Colidem	91
Git Integration com CMD: Controlando o Caos	95
Docker Integration com CMD: Containers de Caos	100
Cloud Integration com CMD: Problemas em Escala Global	106
PowerShell: A Evolução do Terminal Windows	111
Introdução ao PowerShell	118
Conceitos Básicos do PowerShell	124
Cmdlets: Os Comandos do PowerShell	131
Pipeline: O Poder do Encadeamento	138
Objetos no PowerShell	145
PowerShell Avançado	153
Scripting no PowerShell	160
Módulos PowerShell	168
Tratamento de Erros no PowerShell	172
PowerShell Remoting	177
Segurança no PowerShell	183
Debugging no PowerShell	188

Exercícios PowerShell	193
PowerShell Integration	201
Azure PowerShell Integration	207
AWS PowerShell Integration	214
PowerShell DevOps Integration	221
Comparação de Terminais	228
CMD vs PowerShell: Duelo de Terminais	231
Windows vs Unix Terminals	234
Alternativas de Terminal	238
Referência Completa	243
Referência CMD	245
Referência PowerShell	250
Glossário	255
Melhores Práticas	262
Troubleshooting Guide	267
Terminal Cheat Sheet	272

Terminal Windows: Guia do Desenvolvedor



! "Em um mundo onde código é lei, o terminal é sua arma."

O que você vai encontrar aqui

Este guia foi feito por devs, para devs. Sem rodeios, direto ao ponto. Vamos dominar as duas principais ferramentas de linha de comando do Windows:

- **CMD:** O clássico prompt de comando
- **PowerShell:** A ferramenta moderna de automação

Navegação Rápida

CMD - O Básico do Underground

```
C:\> help
# Seu primeiro passo no submundo dos comandos
```

- Comandos Essenciais ([Comandos Básicos: Seu Arsenal Inicial](#))
- Manipulação de Arquivos ([Operações com Arquivos: Dominando o Sistema](#))
- Scripts Batch ([Scripts Batch: Automatize ou Morra Tentando](#))

PowerShell - O Arsenal Avançado

```
PS C:\> Get-Command  
# Poder é conhecimento
```

- Cmdlets Fundamentais ([Cmdlets: Os Comandos do PowerShell](#))
- Pipeline & Objetos ([Pipeline: O Poder do Encadeamento](#))
- Automação & Scripts ([Scripting no PowerShell](#))

Por que este guia?

- ✓ Exemplos práticos do mundo real
- ✓ Foco em desenvolvimento e automação
- ✓ Sem enrolação, direto ao código
- ✓ Truques e hacks que você vai usar todo dia

Comece Aqui

Para Iniciantes

```
# Primeiro, aprenda a navegar  
cd /d C:\seu\projeto  
dir /b
```

Para Devs Experientes

```
# Automatize seu workflow  
Get-ChildItem -Recurse -Include *.js |  
    Where-Object { $_.Length -gt 1MB }
```

Projetos Práticos

1. **Build Automático:** Scripts para compilar e deployar

2. **Git Hooks:** Automatização de commits e pushes

3. **Monitoramento:** Scripts para logs e performance

Contribua

Encontrou um bug? Tem uma dica ninja? Abra uma issue

(<https://github.com/mrpunkdasilva/Learn-Windows-Terminals/issues>) ou envie um PR.

Dica do Dia

> Pressione Tab para autocompletar. Seu tempo é valioso demais
> para digitar caminhos completos.

"O terminal é mais do que uma ferramenta. É uma extensão do seu código."

Começando com Terminais Windows

```

  _ _ _ _ _ _ _ _ _ _
 / _ | _ | _ | _ | _ | \ | | / _ |
 | | _ | _ | | | | | | | \ | | | _
 | | _ | | _ | | | | | | | \ | | _ |
 \ _ | _ | | | | | | _ | _ | \ _ |

  _ _ _ _ _ _ _ _ _ _
 / _ | _ | _ | / \ | _ \ _ | _ \
 \ _ \ | | / _ \ | | ) | | | _ | | |
 _ ) | | | / _ \ | _ < | | | _ | | |
 | _ / | _ / \ _ \ | \ _ \ | | _ | _ /

```

 "O começo de toda jornada começa com um simples comando."

Pré-requisitos

- Windows 10/11
- Permissões de administrador (opcional)
- Vontade de aprender
- ☕ Café (muito café)

Escolhendo seu Terminal

CMD

```
:: Para iniciantes
cmd.exe
```

- Interface clássica
- Presente em todo Windows
- Ótimo para começar

PowerShell

```
# Para usuários avançados  
pwsh.exe
```

- Mais poderoso
- Orientado a objetos
- Melhor para automação

Primeiros Passos

1. Abrindo o Terminal

Métodos rápidos:

- Windows + R, digite `cmd` ou `powershell`
- Windows + X, selecione Terminal
- Busque "Terminal" no menu iniciar

2. Configuração Básica

```
:: CMD - Configuração inicial  
chcp 65001           :: UTF-8  
color 0A             :: Verde sobre preto  
mode con: cols=120   :: Largura confortável
```

```
# PowerShell - Configuração inicial  
Set-ExecutionPolicy RemoteSigned
```

```
Install-Module PSReadLine
```

3. Comandos Essenciais

Navegação

```
:: Comandos universais
cd                # Mostra diretório atual
dir              # Lista arquivos
cls              # Limpa tela
help             # Mostra ajuda
```

Sistema

```
:: Informações básicas
systeminfo       # Info do sistema
tasklist         # Processos ativos
ipconfig         # Config de rede
```

Estrutura do Curso

Fase 1: Fundamentos

1. CMD Básico ([CMD: Seu Primeiro Terminal](#))
2. PowerShell Básico ([Introdução ao PowerShell](#))
3. Navegação ([Navegação no CMD: Perdido no Terminal](#))

Fase 2: Prática

1. Exercícios CMD ([Exercícios CMD: Hora de Praticar](#))
2. Exercícios PowerShell ([Exercícios PowerShell](#))
- 3.

Dicas para Iniciantes

Boas Práticas

- Comece com CMD
- Pratique diariamente
- Mantenha anotações
- Use o `help` frequentemente
- Não tenha medo de errar

Evite

- Copiar sem entender
- Pular conceitos básicos
- Executar scripts desconhecidos
- Ignorar mensagens de erro

Ferramentas Recomendadas

Essenciais

- Windows Terminal
- Visual Studio Code
- Git Bash
- Notepad++

Opcionais

- ConEmu
- Cmder
- Terminal-Icons

- Oh My Posh

Próximos Passos

1. Introdução ao CMD ([CMD: Seu Primeiro Terminal](#))

- Comandos básicos
- Navegação
- Scripts simples

2. Introdução ao PowerShell ([Introdução ao PowerShell](#))

- Cmdlets básicos
- Pipeline
- Objetos

3. Comparação de Terminais ([Comparação de Terminais](#))

- Quando usar cada um
- Vantagens e desvantagens
- Casos de uso

Recursos Adicionais

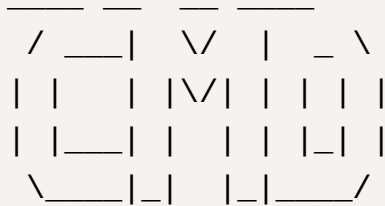
- Documentação Microsoft (<https://docs.microsoft.com>)
- Learn PowerShell (<https://learn.microsoft.com/powershell>)
- CMD Reference (<https://ss64.com/nt>)
- PowerShell Gallery (<https://www.powershellgallery.com>)

Checklist do Iniciante

- ☐ Instalar Windows Terminal
- ☐ Configurar ambiente básico
- ☐ Aprender comandos essenciais
- ☐ Completar exercícios iniciais
- ☐ Criar primeiro script

"Todo expert já foi iniciante. O segredo é começar."

Command Prompt (CMD): O Terminal Clássico do Windows



⚠ "Às vezes o mais antigo é o mais confiável."

Visão Geral

O Command Prompt (CMD) é a interface de linha de comando tradicional do Windows. Apesar de sua idade, continua sendo uma ferramenta essencial para:

- Automação de tarefas
- Diagnóstico de problemas
- Gerenciamento de sistema
- Desenvolvimento de software
- Scripts batch

Por que Usar CMD em 2025?

Vantagens

- Baixo consumo de recursos
- Presente em todo Windows
- Compatibilidade histórica

- Execução rápida
- Scripts batch simples

Limitações

- Sintaxe antiga
- Menos recursos que PowerShell
- Comandos limitados
- Case-insensitive

Estrutura do Curso

Fundamentos

1. Introdução ([CMD: Seu Primeiro Terminal](#))
2. Comandos Básicos ([Comandos Básicos: Seu Arsenal Inicial](#))
3. Navegação ([Navegação no CMD: Perdido no Terminal](#))

Intermediário

1. Operações com Arquivos ([Operações com Arquivos: Dominando o Sistema](#))
2. Comandos do Sistema ([Comandos do Sistema: Controlando o Windows](#))
3. Batch Scripting ([Scripts Batch: Automatize ou Morra Tentando](#))

Avançado

1. Automação ([Automação Avançada: Além dos Scripts](#))
2. Networking ([Networking no CMD: Dominando a Rede](#))
3. Debugging ([Debugging no CMD: Onde os Erros Vão Para Festa](#))

Ambiente de Desenvolvimento

Configuração Recomendada

```
:: Configurações básicas
chcp 65001          :: UTF-8
prompt $P$G         :: Prompt personalizado
color 0A            :: Cores (verde sobre preto)
mode con: cols=120 lines=3000 :: Tamanho da janela
```

Ferramentas Complementares

- Windows Terminal
- ConEmu
- Cmder
- Git Bash

Quick Reference

Comandos Essenciais

```
:: Navegação
cd          :: Mostra/muda diretório
dir         :: Lista arquivos
path       :: Mostra/define PATH

:: Sistema
tasklist    :: Lista processos
systeminfo  :: Info do sistema
ipconfig    :: Config de rede

:: Arquivos
copy        :: Copia arquivos
```

```
move      :: Move arquivos
del       :: Deleta arquivos
```

Atalhos do Teclado

Atalho	Função
↑/↓	Histórico
Tab	Autocompletar
F7	Histórico visual
Ctrl+C	Cancelar

Integração com Ferramentas

DevOps

- Git
- Docker
- Cloud CLIs

Build Tools

- MSBuild
- Maven
- npm

Melhores Práticas

Organização

- Scripts em pasta dedicada
- Nomenclatura consistente
- Documentação inline
- Logs estruturados

Segurança

- Validação de inputs
- Escape de caracteres
- Privilégios mínimos
- Tratamento de erros

Próximos Passos

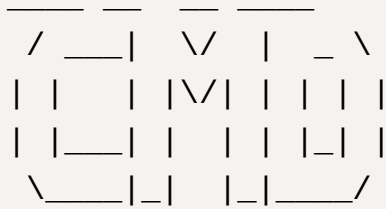
1. Introdução ao CMD ([CMD: Seu Primeiro Terminal](#))
2. Comandos Básicos ([Comandos Básicos: Seu Arsenal Inicial](#))
3. PowerShell ([PowerShell: A Evolução do Terminal Windows](#)) (Quando precisar mais poder)

Recursos Adicionais

- Documentação Microsoft (<https://docs.microsoft.com/windows-server/administration/windows-commands/windows-commands>)
- SS64 CMD Reference (<https://ss64.com/nt/>)
- Batch Script Tutorial (https://www.tutorialspoint.com/batch_script/index.htm)

"CMD: Porque às vezes o básico é tudo que você precisa."

CMD: Seu Primeiro Terminal



⚠ "Antes do PowerShell, antes das GUIs, havia o CMD. E ele ainda tem seu lugar no arsenal do desenvolvedor moderno."

Por que aprender CMD em 2025?

O Command Prompt (CMD) continua sendo uma ferramenta essencial no ambiente Windows por várias razões:

- **Velocidade:** Execução de comandos muito mais rápida que interfaces gráficas
- **Scripts Batch:** Amplamente utilizados em sistemas legados e processos de build
- **Troubleshooting:** Fundamental para diagnóstico e resolução de problemas
- **Base Sólida:** Entender CMD facilita a transição para PowerShell
- **Compatibilidade:** Funciona em qualquer versão do Windows sem dependências
- **Recursos:** Consome menos memória que terminais modernos

Acessando o CMD

Existem várias formas de acessar o CMD:

```
:: Método 1 - Run Dialog  
Windows + R  
cmd  
Enter
```

```
:: Método 2 - Menu Iniciar  
Windows + X -> Command Prompt (Admin)
```

```
:: Método 3 - File Explorer  
Alt + D  
cmd  
Enter
```

Configuração para Desenvolvimento

Configurações Básicas

```
:: Habilitar recursos avançados  
reg add "HKEY_CURRENT_USER\Console" /v QuickEdit /t REG_DWORD /d 1  
/f  
reg add "HKEY_CURRENT_USER\Console" /v InsertMode /t REG_DWORD /d  
1 /f  
reg add "HKEY_CURRENT_USER\Console" /v ScreenBufferSize /t  
REG_DWORD /d 0x23290096 /f  
reg add "HKEY_CURRENT_USER\Console" /v FontSize /t REG_DWORD /d  
0x00100008 /f  
reg add "HKEY_CURRENT_USER\Console" /v FontFamily /t REG_DWORD /d  
0x00000036 /f  
reg add "HKEY_CURRENT_USER\Console" /v FontWeight /t REG_DWORD /d  
0x00000190 /f
```

Personalização Avançada

```
:: Cores e aparência  
reg add "HKEY_CURRENT_USER\Console" /v ColorTable00 /t REG_DWORD  
/d 0x00000000 /f  
reg add "HKEY_CURRENT_USER\Console" /v ColorTable07 /t REG_DWORD  
/d 0x00cccccc /f
```

```
reg add "HKEY_CURRENT_USER\Console" /v WindowSize /t REG_DWORD /d  
0x00320078 /f
```

Comandos Essenciais para Desenvolvimento

Navegação e Gerenciamento de Projetos

```
:: Navegação rápida  
cd /d D:\projetos                :: Muda drive e diretório  
dir /b /s *.js                   :: Lista recursivamente arquivos JS  
tree /f                          :: Mostra estrutura de diretórios  
pushd \\servidor\share           :: Salva localização atual e navega  
popd                             :: Retorna à localização salva  
  
:: Gerenciamento de arquivos  
xcopy src dest /E /I /H          :: Cópia preservando estrutura  
robocopy source dest /MIR        :: Sincronização espelhada  
attrib +h node_modules          :: Oculta pasta
```

Processos e Rede

```
:: Gerenciamento de processos  
tasklist | findstr "node"        :: Lista processos Node.js  
taskkill /F /IM node.exe         :: Força fechamento do Node  
wmic process where "name='node.exe'" get commandline :: Detalhes  
  
:: Rede e portas  
netstat -ano | findstr "3000"    :: Verifica porta específica  
ipconfig /flushdns               :: Limpa cache DNS  
nslookup api.exemplo.com        :: Resolve DNS
```

Dicas Práticas

Aliases e Paths Personalizados

```

:: Criar arquivo autoexec.bat
@echo off
:: Aliases úteis
doskey ls=dir /b $*
doskey clear=cls
doskey ..=cd ..
doskey ...=cd ../..
doskey gs=git status
doskey gl=git log --oneline
doskey gp=git pull
doskey npm-clean=rmdir /s /q node_modules && del package-lock.json

:: Paths de desenvolvimento
set PATH=%PATH%;C:\dev\tools
set JAVA_HOME=C:\Program Files\Java\jdk-17
set MAVEN_HOME=C:\dev\apache-maven

```

Debugging Rápido

```

:: Verificação de portas
:checkPort
set PORT=%1
netstat -ano | findstr ":%PORT%"
if errorlevel 1 (
    echo Porta %PORT% livre
) else (
    echo Porta %PORT% em uso
    for /f "tokens=5" %%a in ('netstat -ano ^| findstr ":%PORT%")')
do (
    echo PID: %%a
    tasklist | findstr "%%a"
)
)

:: Matar processo
:killProcess
set PORT=%1

```

```
for /f "tokens=5" %%a in ('netstat -ano ^| findstr ":%PORT%") do
(
    taskkill /F /PID %%a
    if errorlevel 1 (
        echo Falha ao matar processo
    ) else (
        echo Processo finalizado
    )
)
```

Atalhos do Teclado Essenciais

Atalho	Função	Contexto
F7	Histórico de comandos	Mostra lista navegável
Alt + F7	Limpa histórico	Útil para segurança
F3	Repete último comando	Digitação rápida
Alt + Enter	Modo tela cheia	Melhor visualização
Ctrl + C	Cancela comando	Interrompe execução
Ctrl + Break	Força interrupção	Quando Ctrl+C falha

Próximos Passos

1. Comandos Básicos ([Comandos Básicos: Seu Arsenal Inicial](#))

- Aprenda os comandos fundamentais
- Pratique navegação e manipulação

2. Operações com Arquivos ([Operações com Arquivos: Dominando o Sistema](#))

- Gerenciamento avançado de arquivos
- Automação de backups

3. Scripts Batch ([Scripts Batch: Automatize ou Morra Tentando](#))

- Crie seus primeiros scripts
- Automatize tarefas repetitivas

Recursos Adicionais

- Documentação Oficial Microsoft (<https://docs.microsoft.com/windows-server/administration/windows-commands/windows-commands>)
- SS64 CMD Reference (<https://ss64.com/nt/>)
- Rob van der Woude's Scripting Pages (<https://www.robvanderwoude.com/batchstart.php>)

Troubleshooting Comum

Problema	Solução	Comando
Acesso Negado	Executar como Admin	<code>runas /user:Administrator cmd</code>
Codificação	Mudar para UTF-8	<code>chcp 65001</code>
Path Longo	Usar subst	<code>subst X: C:\caminho\muito\longo</code>
Erro de Sintaxe	Verificar aspas	<code>echo "Uso correto de aspas"</code>

"O terminal é a interface mais poderosa que você pode dominar."

Comandos Básicos: Seu Arsenal Inicial

/ ____| |
 | | ____ _ _ _ _ _ _ _ _ _ _ | | ____ _
 | | / _ \ | ' _ ` \_ \ / \_ ` | ' _ \ | ' _ \ / _ \
 | | ____ | (-) | | | | | | (- | | | | | | (-) | (-) |
 ________/_|_| |_| |_| __,/__| |_| |_|._/ __/_

⚠ "Conhecimento é poder. Poder é controle. Controle é tudo."

Navegação no Sistema

```
:: Navegação básica
cd          :: Mostra diretório atual
cd ..       :: Volta um nível
cd /d D:     :: Muda drive e diretório
pushd       :: Salva diretório atual na pilha
popd        :: Retorna ao último diretório salvo
```

Listagem e Busca

```
:: Listagem avançada
dir /a          :: Mostra arquivos ocultos
dir /s          :: Lista recursivamente
dir /b /s       :: Formato limpo e recursivo

:: Busca ninja
where python     :: Localiza executável
findstr /s /i TODO :: Busca TODO em todos arquivos
```

Manipulação de Arquivos

```
:: Operações básicas
copy src.txt dst.txt      :: Copia arquivo
move old.* new\           :: Move arquivos
del *.tmp                 :: Deleta temporários
ren old.js new.js         :: Renomeia arquivo

:: Operações avançadas
robocopy src dst /MIR     :: Sincronização espelho
xcopy /s /e /h            :: Copia preservando atributos
```

Processos e Rede

```
:: Gerenciamento de processos
tasklist | findstr "node"  :: Lista processos node
taskkill /IM node.exe /F   :: Mata processo
start "" "http://localhost" :: Abre navegador

:: Comandos de rede
netstat -ano               :: Portas em uso
ipconfig /all              :: Config de rede
ping -t localhost          :: Ping contínuo
```

Informações do Sistema

```
:: Debug e diagnóstico
systeminfo | findstr "Memory" :: Info de memória
wmic cpu get name             :: Info do processador
ver                           :: Versão do Windows
```

Pipes e Redirecionamento

```
:: Manipulação de output
dir > lista.txt           :: Salva em arquivo
type file.txt | sort      :: Ordena conteúdo
findstr "erro" log.txt    :: Filtra conteúdo
```

Scripts Rápidos para Dev

```
:: Limpeza de builds
@echo off
for /d /r . %%d in (node_modules) do @if exist "%%d" rmdir /s /q "%%d"
for /d /r . %%d in (dist) do @if exist "%%d" rmdir /s /q "%%d"

:: Monitor de porta
:check
netstat -ano | findstr ":3000"
timeout /t 5
goto check
```

Atalhos de Produtividade

Comando	Descrição	Uso Dev
<code>doskey h=history</code>	Alias para histórico	Reutilizar comandos
<code>cls</code>	Limpa tela	Manter terminal limpo
<code>echo %PATH%</code>	Mostra path	Debug de ambiente

Dicas Avançadas

```
:: Execução silenciosa
start /b programa.exe > nul 2>&1
```

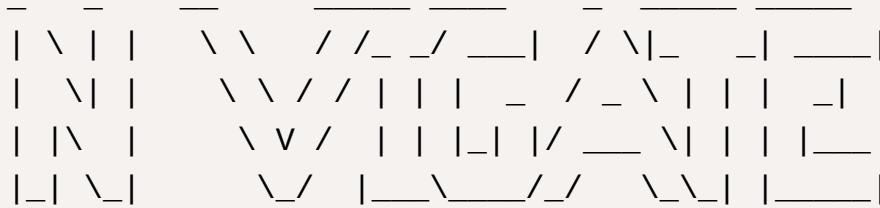
```
:: Captura de erros  
comando.exe 2> erro.log  
  
:: Execução condicional  
comando1.exe && comando2.exe || comando3.exe
```

Próximos Passos

1. Operações com Arquivos ([Operações com Arquivos: Dominando o Sistema](#))
2. Scripts Batch ([Scripts Batch: Automatize ou Morra Tentando](#))
3. Networking Avançado ([Networking no CMD: Dominando a Rede](#))

"Mestre dos comandos, mestre do sistema."

Navegação no CMD: Perdido no Terminal



⚠ "Se você nunca se perdeu no sistema de arquivos, você não está explorando o suficiente."

Comandos Básicos de Navegação

Localização Atual

```
:: Onde estou?
cd                :: Mostra diretório atual
echo %CD%         :: Alternativa para ver caminho atual
chdir             :: Outro comando para mostrar diretório
```

Movimentação Básica

```
:: Navegação essencial
cd ..          :: Volta um nível
cd \           :: Vai para raiz do drive atual
cd /d D:       :: Muda drive e diretório
cd "Pasta Com Espacos"  :: Navega para pasta com espaços
```

Técnicas Avançadas

Stack de Diretórios

```
:: Salvando e recuperando localizações
pushd C:\Projetos    :: Salva local atual e vai para C:\Projetos
popd                :: Retorna ao último local salvo
pushd \\servidor\share :: Mapeia rede e navega
```

Navegação Rápida

```
:: Atalhos úteis
cd -                :: Alterna entre últimos diretórios
cd ..\..\..\        :: Volta múltiplos níveis
cd %USERPROFILE%    :: Vai para pasta do usuário
cd %TEMP%           :: Acessa pasta temporária
```

Listagem e Busca

Listagem de Diretórios

```
:: Visualização de conteúdo
dir                :: Lista arquivos e pastas
dir /a            :: Mostra itens ocultos
dir /b            :: Formato simples
dir /s            :: Lista recursiva
dir /p            :: Pausa por página
```

Busca de Arquivos

```
:: Localização de itens
where python.exe   :: Encontra executável no PATH
dir /s /b *.txt     :: Busca recursiva de .txt
forfiles /p C:\ /m *.doc /s :: Busca avançada
```

Truques e Dicas

Navegação com Wildcards

```
:: Usando padrões
cd *proj*           :: Entra na primeira pasta que contém "proj"
dir *2023*          :: Lista itens com "2023" no nome
cd /d D:\*\src      :: Navega para pasta src em qualquer
subdiretório
```

Aliases e Atalhos

```
:: Criando atalhos
doskey home=cd %USERPROFILE%
doskey back=cd ..
doskey ls=dir /b
doskey up=cd ..
```

Mapeamento de Rede

Conexões de Rede

```
:: Mapeamento de unidades
net use Z: \\servidor\share    :: Mapeia compartilhamento
subst X: C:\Projetos\Atual     :: Cria drive virtual
net use * /delete             :: Remove todos mapeamentos
```

Troubleshooting

Problemas Comuns

```
:: Resolvendo erros
cd.           :: Corrige "Invalid directory"
cd /d .       :: Força atualização do prompt
chdir /d %path% :: Reseta contexto do diretório
```

Dicas de Produtividade

Autocompletar

- **Tab**: Completa nomes de arquivos/pastas
- **F7**: Mostra histórico de comandos
- **↑/↓**: Navega pelo histórico
- **Ctrl+C**: Cancela comando atual

Atalhos do Sistema

```
:: Atalhos para locais especiais
cd %APPDATA%           :: Pasta de dados de aplicativos
cd %ProgramFiles%      :: Pasta Program Files
cd %SystemRoot%        :: Pasta do Windows
cd %TEMP%              :: Pasta temporária
```

Checklist de Navegação

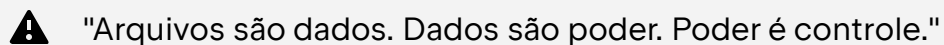
- ☐ Conhecer localização atual
- ☐ Usar comandos básicos com confiança
- ☐ Dominar navegação com wildcards
- ☐ Configurar aliases úteis
- ☐ Mapear drives necessários

Próximos Passos

1. Operações com Arquivos ([Operações com Arquivos: Dominando o Sistema](#))
2. Comandos do Sistema ([Comandos do Sistema: Controlando o Windows](#))
3. Automação ([Automação Avançada: Além dos Scripts](#))

"No CMD, não existem caminhos errados, apenas desvios não planejados."

Operações com Arquivos: Dominando o Sistema

[illegible]

"Arquivos são dados. Dados são poder. Poder é controle."

Operações Essenciais

Criação e Manipulação Básica

```
:: Criação de estrutura
md projeto :: Cria diretório
md "projeto/src" "projeto/tests" :: Múltiplos diretórios
type nul > index.js :: Cria arquivo vazio
copy con config.json :: Cria e edita arquivo
echo console.log() >> index.js :: Append em arquivo
```

```
:: Manipulação básica
copy arquivo.txt backup.txt      :: Copia arquivo
move src\*.js dist\              :: Move arquivos
del *.tmp                        :: Deleta arquivos
ren old.js new.js                :: Renomeia arquivo
```

Operações Avançadas

```
:: Cópia com atributos
xcopy /s /e /h /i src dest    :: Copia preservando tudo
```

```
robocopy src dest /MIR      :: Sincronização espelhada
robocopy src dest /E /X0    :: Cópia apenas mais novos

:: Compactação
compact /c /s *.log         :: Compacta logs
compact /u /s *.*           :: Descompacta tudo
```

Operações em Massa

Processamento em Lote

```
:: Manipulação em lote
for /r %%f in (*.js) do (
    echo Processando: %%f
    :: Substitui texto
    powershell -Command "(Get-Content '%%f') -replace 'const',
'let' | Set-Content '%%f'"
)

:: Renomeação em lote
for %%f in (*.jsx) do (
    set "filename=%%~nf"
    ren "%%f" "!filename!.tsx"
)

:: Processamento condicional
for /r %%f in (*) do (
    if %%~zf gtr 1048576 (
        echo Arquivo grande: %%f
        echo %%~zf bytes
    )
)
```

Backup e Sincronização

Sistema de Backup Completo

```

:: Backup com timestamp
set
"timestamp=%date:~-4%%date:~3,2%%date:~0,2%_%time:~0,2%%time:~3,2%
"
set "backup_dir=backup_%timestamp%"

:: Cria estrutura
md "%backup_dir%"
md "%backup_dir%\code"
md "%backup_dir%\data"

:: Backup de código
xcopy /s /e /h src "%backup_dir%\code"
xcopy /s /e /h config "%backup_dir%\data"

:: Compactação
7z a "%backup_dir%.zip" "%backup_dir%\*"
rmdir /s /q "%backup_dir%"

:: Log
echo Backup realizado em %date% %time% >> backup_log.txt

```

Sincronização de Projetos

```

:: Sincronização bidirecional
:syncProjects
set "source=%~1"
set "target=%~2"

:: Verifica parâmetros
if "%source%"=="" goto :usage
if "%target%"=="" goto :usage

:: Sincroniza
robocopy "%source%" "%target%" /MIR /XD node_modules /XF *.log
if errorlevel 8 goto :syncError

```

```
echo Sincronização concluída
exit /b 0

:syncError
echo Erro na sincronização
exit /b 1

:usage
echo Uso: syncProjects source_dir target_dir
exit /b 1
```

Operações para Desenvolvimento

Limpeza de Projeto

```
:: Limpeza completa
:cleanProject
echo Limpando projeto...

:: Remove builds
rmdir /s /q dist
rmdir /s /q build
rmdir /s /q .next
rmdir /s /q coverage

:: Remove dependências
rmdir /s /q node_modules
del package-lock.json
del yarn.lock

:: Remove caches
del /s /q *.cache
del /s /q .eslintcache
```

Busca de Código

```

:: Busca avançada
:searchCode
set "pattern=%~1"
set "extensions=%~2"

if "%extensions%"==" " set "extensions=js,ts,jsx,tsx"

:: Busca em múltiplos tipos
for %%e in (%extensions:==,%) do (
    echo Buscando em *.%%%e
    findstr /s /i /n /p "%pattern%" *.%%%e
)

:: Conta ocorrências
for %%e in (%extensions:==,%) do (
    findstr /s /i /m "%pattern%" *.%%%e > matches.tmp
    for /f %%a in ('type matches.tmp ^| find /c /v ""') do (
        echo %%a ocorrências em arquivos *.%%%e
    )
)
del matches.tmp

```

Manipulação de Permissões

Gerenciamento de Acesso

```

:: Permissões de arquivo
icacls arquivo.txt /grant Users:F
icacls pasta /grant "Users:(OI)(CI)F"
takeown /f pasta /r /d y

:: Herança
icacls pasta /inheritance:d
icacls pasta /inheritance:e

```

```
:: Remoção de permissões
icacls arquivo.txt /remove Users
```

Scripts Práticos

Monitor de Alterações

```
:: Monitor em tempo real
:watchFiles
set "dir=%~1"
if "%dir%"==" " set "dir=."

echo Monitorando alterações em %dir%
echo Pressione Ctrl+C para sair

:watchLoop
    cls
    echo === %date% %time% ===

    :: Lista arquivos modificados hoje
    forfiles /p "%dir%" /m *.* /d 0 /c "cmd /c echo @file @fdate
@ftime"

    :: Verifica tamanho total
    for /f "tokens=3" %%a in ('dir /-c /w "%dir%"') do set
size=%%a
    echo Total: %size% bytes

    timeout /t 5 > nul
    goto watchLoop
```

Backup Automático

```
:: Sistema de backup incremental
:incrementalBackup
set "source=%~1"
```

```

set "dest=%~2"
set "timestamp=%date:~-4%%date:~3,2%%date:~0,2%"

:: Verifica último backup
if exist "%dest%\last_backup.txt" (
    set /p last_backup=<"%dest%\last_backup.txt"

    :: Copia apenas arquivos novos/modificados
    robocopy "%source%" "%dest%\%timestamp%" /E /XO /XD
node_modules
) else (
    :: Primeiro backup (completo)
    robocopy "%source%" "%dest%\%timestamp%" /E /XD node_modules
)

:: Atualiza registro
echo %timestamp% > "%dest%\last_backup.txt"

```

Boas Práticas

1. Sempre faça backup antes de operações destrutivas
2. Use caminhos absolutos em scripts importantes
3. Verifique permissões antes de operações críticas
4. Mantenha logs de operações importantes
5. Use variáveis para paths frequentemente usados

Troubleshooting

Problema	Causa	Solução
Acesso Negado	Permissões	takeown + icacls
Path Muito Longo	Limite Windows	Usar subst ou \\?\
Arquivo em Uso	Locked	handle.exe para identificar
Caracteres Especiais	Encoding	Usar chcp 65001

Próximos Passos

1. Automação Avançada ([Automação Avançada: Além dos Scripts](#))
2. Scripting Batch ([Scripts Batch: Automatize ou Morra Tentando](#))
3. PowerShell Migration ([Introdução ao PowerShell](#))

"Quem controla os arquivos, controla o sistema."

Comandos do Sistema: Controlando o Windows

 "O sistema é seu. Você só precisa saber como controlá-lo."

Gerenciamento de Processos

Listagem e Monitoramento

```
:: Processos ativos
tasklist                :: Lista todos processos
tasklist /fi "imagename eq chrome.exe" :: Filtra processo
tasklist /v             :: Modo detalhado
wmic process get name,processid,priority :: Info detalhada

:: Monitoramento
tasklist /m              :: Mostra DLLs usadas
tasklist /svc            :: Mostra serviços
wmic process where "name='chrome.exe'" get processid,commandline
```

Controle de Processos

```
:: Manipulação de processos
taskkill /PID 1234      :: Mata por PID
taskkill /IM chrome.exe /F :: Força fechamento
wmic process where name="chrome.exe" delete :: Alternativa
start "" notepad.exe    :: Inicia processo
```

Serviços do Sistema

Gerenciamento de Serviços

```
:: Controle de serviços
sc query                :: Lista serviços
```

```
net start                :: Serviços ativos
net start "Windows Update" :: Inicia serviço
net stop "Print Spooler"  :: Para serviço
sc config service_name start= disabled :: Desabilita
```

Status e Configuração

```
:: Informações de serviços
sc queryex service_name  :: Info detalhada
sc qc service_name       :: Mostra config
wmic service list brief  :: Lista resumida
```

Informações do Sistema

Hardware

```
:: Info de hardware
systeminfo                :: Info completa
wmic cpu get name         :: Processador
wmic memorychip get capacity :: Memória
wmic diskdrive get size,model :: Discos
wmic bios get version     :: BIOS
```

Software e Sistema

```
:: Info de software
ver                        :: Versão Windows
winver                    :: Janela de versão
wmic os get version       :: Versão detalhada
wmic product get name     :: Programas instalados
```

Gerenciamento de Energia

Controle de Energia

```
:: Energia e shutdown
shutdown /s /t 60      :: Desliga em 60s
shutdown /r /t 0       :: Reinicia agora
shutdown /h            :: Hiberna
powercfg /energy        :: Diagnóstico
powercfg /batteryreport :: Relatório bateria
```

Rede e Conectividade

Diagnóstico de Rede

```
:: Ferramentas de rede
ipconfig /all          :: Config completa
netstat -ano           :: Conexões ativas
route print            :: Tabela de rotas
nslookup google.com    :: DNS lookup
tracert 8.8.8.8        :: Trace route
```

Configuração de Rede

```
:: Config de rede
netsh interface show interface :: Interfaces
netsh wlan show networks       :: Redes Wi-Fi
netsh advfirewall set allprofiles state off :: Firewall
```

Scripts de Diagnóstico

Monitor de Sistema

```
@echo off
:monitor
cls
echo === Monitor do Sistema ===
echo Data: %date% Hora: %time%

:: CPU e Memória
```

```
wmic cpu get loadpercentage
wmic OS get FreePhysicalMemory

:: Processos críticos
tasklist | findstr /i "chrome firefox node"

:: Rede
netstat -ano | findstr "ESTABLISHED"

timeout /t 5 > nul
goto monitor
```

Verificação de Saúde

```
:: Script de diagnóstico
@echo off
echo === Diagnóstico do Sistema ===

:: Verificações básicas
sfc /scannow
chkdsk /f
dism /online /cleanup-image /restorehealth

:: Logs de erro
wevtutil qe System /c:5 /f:text
wevtutil qe Application /c:5 /f:text
```

Melhores Práticas

Segurança

- Use `runas` para elevação de privilégios
- Verifique processos desconhecidos
- Monitore portas abertas

- Mantenha logs de alterações

Performance

- Limite processos em background
- Monitore uso de recursos
- Agende manutenção regular
- Documente configurações

Próximos Passos

1. Automação de Sistema ([Automação Avançada: Além dos Scripts](#))
2. Scripting Avançado ([Scripts Batch: Automatize ou Morra Tentando](#))
3. Networking ([Networking no CMD: Dominando a Rede](#))

"Com grandes poderes vêm grandes responsabilidades... e muitos comandos para lembrar."

Técnicas Avançadas de CMD

 "Para os mestres do terminal, onde cada comando conta uma história."

Manipulação Avançada de Strings

Substrings e Parsing

```
:: Extração de substrings
set str=Hello World
set sub=%str:~0,5%      :: Retorna "Hello"
set sub=%str:~6%        :: Retorna "World"
set sub=%str:~-5%        :: Retorna "World"

:: Substituição de strings
set result=%str:Hello=Hi% :: Substitui "Hello" por "Hi"
set result=%str:o=0%      :: Substitui todos "o" por "0"
```

Processamento de Texto

```
:: Manipulação avançada
for /f "tokens=1,2 delims=," %%a in (data.csv) do (
    echo First: %%a, Second: %%b
)

:: Parsing de saída de comando
for /f "tokens=* usebackq" %%i in (`dir /b`) do (
    echo File: %%i
)
```

Controle de Fluxo Avançado

Estruturas Complexas

```

:: Loop aninhado com condicionais
for %%i in (*.txt) do (
    if exist "backup\%%i" (
        echo Arquivo existe
        call :ProcessFile "%%i"
    ) else (
        echo Novo arquivo
        copy "%%i" "backup\%%i"
    )
)

:: Goto dinâmico
set "action=Process"
goto %action%_Section 2>nul || echo Ação inválida

:Process_Section
echo Processando...
exit /b

```

Tratamento de Erros

```

:: Manipulação de erros
setlocal EnableExtensions EnableDelayedExpansion
set "err_level=0"

call :RiskyFunction || (
    set "err_level=!errorlevel!"
    goto :ErrorHandler
)

:ErrorHandler
echo Erro %err_level% ocorreu
exit /b %err_level%

```

Funções Avançadas

Biblioteca de Funções

```
:: Funções reusáveis
:Logger
echo [%date% %time%] %~1 >> log.txt
exit /b

:BackupFile
if not exist "backup" mkdir "backup"
copy "%~1" "backup\%~nx1.bak"
call :Logger "Backup criado: %~1"
exit /b

:GetFileSize
set "size="
for %%A in ("%~1") do set "size=%%~zA"
exit /b
```

Callbacks e Eventos

```
:: Sistema de callbacks
set "callback=ProcessResult"
call :%callback% "dados"

:ProcessResult
echo Processando: %~1
exit /b
```

Integração com Sistema

Manipulação de Registry

```
:: Operações de registro
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion"
reg add "HKCU\Environment" /v Path /t REG_EXPAND_SZ /d
```

```
"%path%;C:\NewPath"  
reg export "HKLM\SOFTWARE\MyApp" backup.reg
```

Gerenciamento de Serviços

```
:: Controle avançado de serviços  
sc config MyService start= delayed-auto  
sc failure MyService reset= 86400 actions=  
restart/60000/restart/60000//  
sc triggerinfo MyService start/port/80
```

Automação Avançada

Monitoramento de Eventos

```
:: Monitor de mudanças  
:WatchDir  
dir /b > dir_new.txt  
fc dir_old.txt dir_new.txt > nul  
if errorlevel 1 (  
    call :ProcessChanges  
)  
move /y dir_new.txt dir_old.txt  
timeout /t 5 > nul  
goto WatchDir
```

Tasks Agendadas

```
:: Criação de tarefas  
schtasks /create /tn "MyTask" /tr "C:\script.bat" /sc daily /st  
09:00  
schtasks /change /tn "MyTask" /enable  
schtasks /query /tn "MyTask" /fo list /v
```

Networking Avançado

Diagnóstico de Rede

```
:: Ferramentas avançadas
netsh trace start capture=yes report=yes
netsh interface ipv4 show subinterfaces
netsh advfirewall firewall add rule name="MyApp" dir=in
action=allow
```

Monitoramento de Conexões

```
:: Monitor de conexões
:MonitorConnections
netstat -nao | findstr "ESTABLISHED" > conn_new.txt
fc conn_old.txt conn_new.txt > nul
if errorlevel 1 (
    call :AlertNewConnection
)
move /y conn_new.txt conn_old.txt
timeout /t 10 > nul
goto MonitorConnections
```

Dicas de Performance

Otimização

- Use `setlocal EnableExtensions EnableDelayedExpansion`
- Evite chamadas desnecessárias a comandos externos
- Utilize variáveis em vez de comandos repetidos
- Minimize uso de `goto` em loops

Debug e Profiling

```
:: Timer simples
set "start=%time%"
call :HeavyFunction
```

```
set "end=%time%"  
call :CalcDuration "%start%" "%end%"
```

Próximos Passos

1. Scripting Avançado ([Scripts Batch: Automatize ou Morra Tentando](#))
2. Integração com DevOps ([Integração CMD: Onde Todos os Mundos Colidem](#))
3. PowerShell Migration ([Introdução ao PowerShell](#))

"A verdadeira maestria vem da compreensão dos detalhes."

Scripts Batch: Automatize ou Morra Tentando

```
  _ _ _ _ _ _ _ _ _ _  
| _ _ ) _ _ _ | | _ _ _ | | _  
| _ _ \ / _ _ | | _ / _ _ | ' _ \  
| | _ ) | ( _ | | | | ( _ | | | |  
| _ _ / \ _ , _ | \ _ \ _ _ | _ | | _ |
```

⚠ "Automatização não é futuro, é sobrevivência."

Scripts batch são a espinha dorsal da automação no Windows. Eles permitem automatizar tarefas repetitivas, gerenciar processos e criar fluxos de trabalho complexos usando apenas o prompt de comando.

Estrutura Básica

Todo script batch começa com algumas configurações essenciais:

```
@echo off  
setlocal enabledelayedexpansion  
  
:: Configurações  
set PROJECT_ROOT=C:\projetos  
set LOG_FILE=script.log  
  
:: Seu código aqui  
echo Iniciando script... >> %LOG_FILE%
```

- `@echo off`: Evita que os comandos apareçam durante a execução
- `setlocal enabledelayedexpansion`: Permite uso de variáveis em tempo de execução
- Variáveis de ambiente são definidas com `set`

- Use `>>` para append em logs, `>` para sobrescrever

Variáveis e Ambiente

As variáveis são o coração de qualquer script. Existem dois tipos principais:

```
:: Variáveis do sistema (sempre disponíveis)
echo Sistema: %OS%                :: Windows_NT
echo Usuário: %USERNAME%          :: JohnDoe
echo Temp: %TEMP%                 ::
C:\Users\JohnDoe\AppData\Local\Temp

:: Variáveis customizadas (definidas por você)
set VERSION=1.0.0                 :: Simples
set /p USER_INPUT=Digite seu nome:  :: Input do usuário
set "COMPLEX_VAR=Valor com espaços" :: Com espaços
set /a RESULT=(%NUM1% + %NUM2%) * 2  :: Operações
matemáticas
```

Dicas de Variáveis

- Use `set /a` para operações matemáticas
- Envolver valores com espaços em aspas
- `%~dp0` representa o diretório do script atual

Estruturas de Controle

O controle de fluxo permite decisões e repetições:

```
:: Condicionais avançados
if exist "package.json" (
    echo Projeto Node.js detectado
    call npm install
) else if exist "pom.xml" (
    echo Projeto Maven detectado
    call mvn install
```

```

) else (
    echo Tipo de projeto desconhecido
    exit /b 1
)

:: Loops poderosos
:: Loop em arquivos
for %%i in (*.js) do (
    echo Processando: %%i
    call node %%i
)

:: Loop em diretórios
for /d %%d in (*) do (
    echo Verificando diretório: %%d
    if exist "%%d\package.json" call :processNodeProject "%%d"
)

:: Loop com números
for /l %%n in (1,1,5) do (
    echo Tentativa %%n de 5
    call :attemptConnection || (
        if %%n equ 5 exit /b 1
    )
)

```

Funções e Módulos

Organize seu código em funções reutilizáveis:

```

:: Função com parâmetros
:processProject
set "PROJECT_DIR=%~1"
set "BUILD_TYPE=%~2"

echo Processando %PROJECT_DIR% como %BUILD_TYPE%
pushd %PROJECT_DIR%

```

```

if "%BUILD_TYPE%"=="node" (
    call npm install
    call npm run build
) else if "%BUILD_TYPE%"=="java" (
    call mvn clean install
)

popd
exit /b 0

:: Chamada da função
call :processProject "C:\projetos\meu-app" "node"
if errorlevel 1 echo Falha no processamento

:: Função com retorno de valor
:calculateSum
set /a "RESULT=%~1 + %~2"
exit /b %RESULT%

:: Capturando retorno
call :calculateSum 5 3
echo Resultado: %errorlevel%

```

Scripts para Dev

Ferramentas práticas para desenvolvimento:

```

:: Setup completo de projeto
:initProject
set "PROJECT_NAME=%~1"
set "PROJECT_TYPE=%~2"

mkdir "%PROJECT_NAME%"
cd "%PROJECT_NAME%"

:: Estrutura base

```

```

mkdir src tests docs config
echo node_modules> .gitignore
echo dist>> .gitignore
echo .env>> .gitignore

:: Setup específico por tipo
if "%PROJECT_TYPE%"=="node" (
    call npm init -y
    call npm install --save-dev jest eslint prettier
    copy nul src\index.js
    copy nul tests\index.test.js
) else if "%PROJECT_TYPE%"=="react" (
    call npx create-react-app .
    call npm install --save-dev @testing-library/react
)

:: Git init
git init
git add .
git commit -m "Initial commit"

echo Projeto %PROJECT_NAME% criado com sucesso!
exit /b 0

:: Watch com hot reload
:devWatch
echo Iniciando modo desenvolvimento...
:watchLoop
    cls
    echo [%time%] Compilando...
    call npm run build

    if errorlevel 1 (
        echo [ERRO] Falha na compilação
        timeout /t 5 /nobreak > nul
    ) else (
        echo [OK] Compilação bem sucedida
    )

```

```
timeout /t 2 /nobreak > nul
goto watchLoop
```

Manipulação de Erros

Sistema robusto de tratamento de erros:

```
:: Sistema completo de erro
setlocal enabledelayedexpansion
set "ERROR_LOG=%~dp0\errors.log"

:: Try-Catch simulado
:try
    set "ERROR_OCCURRED="

    :: Seu código aqui
    call npm install 2>nul || set ERROR_OCCURRED=1

    if defined ERROR_OCCURRED (
        call :catch
        exit /b 1
    )
    goto :finally

:catch
    echo [%date% %time%] Erro durante npm install >> %ERROR_LOG%
    echo Falha na instalação. Tentando recuperar...
    call npm cache clean --force
    call :notifyTeam "Falha no build"
    exit /b 1

:finally
    echo Limpando recursos...
    if exist "temp" rmdir /s /q temp
    exit /b 0
```

```
:: Sistema de notificação
:notifyTeam
set "MESSAGE=%~1"
curl -X POST "https://api.slack.com/webhook" -d "
{'text': '%MESSAGE%'}"
exit /b
```

Integração com DevOps

Automatização de processos de desenvolvimento:

```
:: Pipeline completo
:buildAndDeploy
set "ENV=%~1"
set "VERSION=%~2"

:: Validações
if "%ENV%"==" " set "ENV=dev"
if "%VERSION%"==" " set "VERSION=latest"

:: Backup
set "BACKUP_DIR=backup_%date:~-4%%date:~3,2%%date:~0,2%"
if not exist "%BACKUP_DIR%" mkdir "%BACKUP_DIR%"
xcopy /s /e dist\ "%BACKUP_DIR%"

:: Build
echo [%time%] Iniciando build %VERSION%...
call npm run build
if errorlevel 1 goto :handleError

:: Testes
echo [%time%] Executando testes...
call npm test
if errorlevel 1 goto :handleError

:: Deploy
echo [%time%] Realizando deploy para %ENV%...
```

```

if "%ENV%"=="prod" (
    call :validateProd || goto :handleError
    robocopy dist\ \\prod-server\deploy\ /MIR /MT:8
) else (
    robocopy dist\ \\dev-server\deploy\ /MIR /MT:8
)

echo Deploy concluído com sucesso!
exit /b 0

:handleError
echo [ERROR] Falha no pipeline
call :notifyTeam "Pipeline falhou em %ENV%"
exit /b 1

:validateProd
echo Validando ambiente de produção...
call :checkDiskSpace || exit /b 1
call :checkPermissions || exit /b 1
call :backupDB || exit /b 1
exit /b 0

```

Scripts Práticos

Monitor de Recursos

```

:monitorResources
cls
echo === Monitor de Recursos ===
tasklist | findstr "node"
wmic cpu get loadpercentage
timeout /t 5 > nul
goto monitorResources

```

Limpeza de Ambiente

```
:: Limpa ambiente de desenvolvimento
:cleanDev
rmdir /s /q node_modules
rmdir /s /q dist
del package-lock.json
del yarn.lock
npm cache clean --force
```

Dicas Avançadas

```
:: Debug mode
set DEBUG=1
if defined DEBUG (
    echo [DEBUG] Variável=%VARIÁVEL%
)

:: Captura de output
for /f "tokens=* delims=" %%a in ('git status') do (
    set "GIT_STATUS=%%a"
)
```

Troubleshooting

Soluções para problemas comuns:

Problema	Solução	Exemplo Detalhado
Variável não expande	Use delayed expansion	<code>set "var=123" & echo !var!</code>
Caracteres especiais	Use escape ^	<code>echo ^ ^&^<^></code>
Path com espaço	Use aspas	<code>cd "C:\Program Files\App"</code>
Permissão negada	Execute como admin	<code>runas /user:admin "script.bat"</code>
Codificação UTF-8	Use chcp 65001	<code>chcp 65001 > nul</code>

Próximos Passos

1. Automação Avançada ([Automação Avançada: Além dos Scripts](#)) - Aprenda técnicas avançadas de automação
2. PowerShell Migration ([Introdução ao PowerShell](#)) - Evolua seus scripts para PowerShell
3. - Integre com pipelines modernos

Recursos Adicionais

- Documentação Oficial do Windows (<https://docs.microsoft.com/windows-server/administration/windows-commands/windows-commands>)
- Batch Script Tutorial (https://www.tutorialspoint.com/batch_script/index.htm)
- Fórum da Comunidade (<https://stackoverflow.com/questions/tagged/batch-file>)

"Automatize hoje o trabalho de amanhã."

Variáveis de Ambiente: O Coração do Sistema

⚠ "Variáveis de ambiente são a alma do sistema operacional, definindo seu comportamento e capacidades."

Fundamentos

Variáveis do Sistema

```
:: Variáveis comuns do sistema
echo %SYSTEMROOT%      :: Pasta do Windows
echo %USERPROFILE%     :: Pasta do usuário
echo %APPDATA%         :: Dados de aplicativos
echo %PATH%            :: Caminhos executáveis
echo %COMPUTERNAME%    :: Nome do computador
```

Variáveis de Usuário

```
:: Variáveis personalizadas
setx MY_VAR "valor"     :: Permanente
set TEMP_VAR=valor      :: Temporária
set "COMPLEX_VAR=valor com espaços"
```

Manipulação de Variáveis

Operações Básicas

```
:: Criação e modificação
set PATH=%PATH%;C:\NovoPath :: Append ao PATH
set VAR=                    :: Remove variável
```

```
set VAR_NAME          :: Lista variável
set                   :: Lista todas
```

Expansão Delayed

```
:: Uso de !variável!
setlocal EnableDelayedExpansion
set VAR=Inicial
for %%i in (1 2 3) do (
    set VAR=Novo%%i
    echo !VAR!
)
endlocal
```

Variáveis Especiais

Variáveis de Data e Hora

```
:: Data e hora
echo %DATE%          :: Data atual
echo %TIME%          :: Hora atual
set
timestamp=%date:~-4%%date:~3,2%%date:~0,2%_%time:~0,2%%time:~3,2%
```

Variáveis de Argumentos

```
:: Argumentos de script
echo %0              :: Nome do script
echo %1              :: Primeiro argumento
echo %*              :: Todos argumentos
shift                :: Desloca argumentos
```

Escopo e Persistência

Escopo Local vs Global

```
:: Controle de escopo
setlocal
set LOCAL_VAR=valor
endlocal

:: Preservando valores
setlocal
set RESULT=valor
endlocal & set RESULT=%RESULT%
```

Persistência

```
:: Variáveis permanentes
setx PATH "%PATH%;C:\NovoPath" /M :: Sistema
setx USER_VAR "valor" :: Usuário
reg add "HKCU\Environment" /v MyVar /t REG_SZ /d "valor"
```

Variáveis para Desenvolvimento

Configuração de Ambiente Dev

```
:: Setup desenvolvimento
set JAVA_HOME=C:\Program Files\Java\jdk-17
set MAVEN_HOME=C:\apache-maven
set NODE_HOME=C:\Program Files\nodejs
set PATH=%PATH%;%JAVA_HOME%\bin;%MAVEN_HOME%\bin;%NODE_HOME%
```

Variáveis para Build

```
:: Variáveis de build
set BUILD_VERSION=1.0.0
set BUILD_ENV=development
set DEBUG=true
set LOG_LEVEL=verbose
```

Técnicas Avançadas

Manipulação de Strings

```
:: Operações com strings
set STR=Hello World
echo %STR:~0,5%           :: Substring (Hello)
echo %STR:Hello=Hi%      :: Substituição
echo %STR:o=0%           :: Replace global
```

Variáveis Dinâmicas

```
:: Nomes dinâmicos
set PREFIX=TEST
set %PREFIX%_1=Valor1
set %PREFIX%_2=Valor2
call set DYNAMIC=%%%PREFIX%_1%%
```

Debug e Troubleshooting

Diagnóstico

```
:: Ferramentas de debug
set                               :: Lista todas variáveis
echo %ErrorLevel%                :: Último código de erro
set VAR 2>nul                    :: Verifica existência
if defined VAR (echo Existe)
```

Logging

```
:: Log de variáveis
echo [%date% %time%] VAR=%VAR% >> vars.log
set > snapshot.txt              :: Dump de variáveis
comp snapshot1.txt snapshot2.txt :: Compara estados
```

Melhores Práticas

Segurança

- Evite armazenar senhas em variáveis
- Use `setx` com cautela
- Valide valores antes de usar
- Limpe variáveis sensíveis após uso

Organização

- Use prefixos consistentes
- Documente variáveis importantes
- Mantenha backup das configurações
- Agrupe variáveis relacionadas

Próximos Passos

1. Batch Scripting ([Scripts Batch: Automatize ou Morra Tentando](#))
2. Automação ([Automação Avançada: Além dos Scripts](#))
3. Integração DevOps ([Integração CMD: Onde Todos os Mundos Colidem](#))

"As variáveis de ambiente são o DNA do seu sistema - cuide bem delas."

Networking no CMD: Dominando a Rede

⚠ "A rede é como um labirinto: você precisa conhecer os caminhos certos para não se perder."

Comandos Básicos de Rede

Configuração IP

```
:: Informações de rede
ipconfig                :: Info básica
ipconfig /all           :: Info detalhada
ipconfig /release       :: Libera IP
ipconfig /renew          :: Renova IP
ipconfig /flushdns       :: Limpa cache DNS
```

Diagnóstico de Conectividade

```
:: Testes básicos
ping google.com         :: Teste básico
ping -t 8.8.8.8         :: Ping contínuo
tracert github.com      :: Trace route
pathping servidor       :: Path + ping
nslookup dominio.com    :: Consulta DNS
```

Monitoramento de Rede

Conexões Ativas

```
:: Monitoramento de portas
netstat -an             :: Todas conexões
netstat -b              :: Processos + portas
```

```
netstat -ano          :: PIDs
netstat -sp TCP       :: Estatísticas TCP
```

Análise de Tráfego

```
:: Captura de pacotes
netsh trace start capture=yes    :: Inicia captura
netsh trace stop                :: Para captura
netsh trace show status         :: Status
```

Configuração de Rede

Interface de Rede

```
:: Configuração de interface
netsh interface show interface    :: Lista interfaces
netsh interface ip show config    :: Mostra config
netsh interface ip set address "Local" static IP MASK GATEWAY
netsh interface ip set dns "Local" static DNS_SERVER
```

Firewall

```
:: Gerenciamento do firewall
netsh advfirewall show allprofiles :: Status
netsh advfirewall set allprofiles state on
netsh advfirewall firewall add rule name="App" dir=in action=allow
program="C:\app.exe"
```

Compartilhamento e Recursos

Compartilhamentos

```
:: Gerenciamento de shares
net share          :: Lista shares
net share PASTA=C:\Pasta :: Cria share
```

```
net share PASTA /delete      :: Remove share
net use Z: \\servidor\share  :: Mapeia drive
```

Recursos de Rede

```
:: Recursos disponíveis
net view                  :: Lista recursos
net view \\servidor      :: Recursos do servidor
net session              :: Sessões ativas
net use                  :: Conexões ativas
```

Troubleshooting

Diagnóstico Avançado

```
:: Ferramentas avançadas
route print              :: Tabela de rotas
arp -a                  :: Cache ARP
netsh wlan show networks :: Redes Wi-Fi
netsh diag show adapter  :: Diagnóstico
```

Script de Diagnóstico

```
@echo off
echo === Diagnóstico de Rede ===
ipconfig /all
echo.
echo === Teste de Conectividade ===
ping -n 1 8.8.8.8
ping -n 1 google.com
echo.
echo === Portas Ativas ===
netstat -an | findstr "ESTABLISHED"
```

Automação de Rede

Monitor de Conexão

```
:: Monitor de conectividade
@echo off
:check
cls
echo Monitorando conexão...
ping -n 1 8.8.8.8 | findstr "TTL=" > nul
if errorlevel 1 (
    echo Conexão perdida!
    echo %date% %time% - Falha >> network.log
) else (
    echo Conexão OK
)
timeout /t 5 > nul
goto check
```

Reset de Rede

```
:: Script de reset
@echo off
echo Resetando configurações de rede...
ipconfig /release
ipconfig /flushdns
netsh winsock reset
netsh int ip reset
ipconfig /renew
echo Reset completo. Reinicie o computador.
```

Melhores Práticas

Segurança

- Use firewall adequadamente
- Monitore conexões suspeitas

- Mantenha logs de atividade
- Limite compartilhamentos

Performance

- Otimize configurações TCP/IP
- Monitore latência
- Gerencie largura de banda
- Mantenha DNS atualizado

Próximos Passos

1. Segurança de Rede ([Segurança no CMD: Porque Paranoia é Apenas Bom Senso](#))
2. Automação Avançada ([Automação Avançada: Além dos Scripts](#))
3. Integração com Cloud ([Cloud Integration com CMD: Problemas em Escala Global](#))

"Na rede, cada comando é uma ponte para algum lugar."

Automação Avançada: Além dos Scripts

[illegible]

! "A verdadeira produtividade está na automação inteligente."

Automação de Build

```
:: Build automatizado
@echo off
setlocal enabledelayedexpansion

:: Configurações
set BUILD_DIR=build
set SRC_DIR=src
set
TIMESTAMP=%date:~-4%%date:~-3,2%%date:~0,2%_%time:~0,2%%time:~3,2%

:: Limpeza
rmdir /s /q %BUILD_DIR%
mkdir %BUILD_DIR%

:: Build
call npm run build
if errorlevel 1 (
    echo Build falhou
    exit /b 1
)
```

```
)
```

```
:: Arquivamento
```

```
7z a %BUILD_DIR%\build_%TIMESTAMP%.zip %BUILD_DIR%\*
```

Monitoramento Automatizado

```
:: Monitor de sistema
```

```
:monitor
```

```
cls
```

```
echo === Status do Sistema ===
```

```
echo Time: %time%
```

```
:: CPU e Memória
```

```
wmic cpu get loadpercentage
```

```
wmic OS get FreePhysicalMemory
```

```
:: Processos
```

```
tasklist | findstr /i "node npm"
```

```
:: Portas
```

```
netstat -ano | findstr "LISTENING"
```

```
timeout /t 10 > nul
```

```
goto monitor
```

Integração com Git

```
:: Auto-commit e push
```

```
:gitSync
```

```
set BRANCH=main
```

```
set COMMIT_MSG="Auto-sync %date% %time%"
```

```
:: Verifica alterações
```

```
git status | findstr "nothing to commit" > nul
```

```

if not errorlevel 1 (
    echo Nada para sincronizar
    exit /b 0
)

:: Commit e push
git add .
git commit -m %COMMIT_MSG%
git push origin %BRANCH%

```

Deployment Automatizado

```

:: Deploy multi-ambiente
:deploy
set ENV=%1
if "%ENV%"==" " set ENV=dev

:: Configurações por ambiente
if "%ENV%"=="prod" (
    set DEPLOY_DIR=\\prod-server\deploy
    set BACKUP_DIR=\\backup-server\prod
) else (
    set DEPLOY_DIR=\\dev-server\deploy
    set BACKUP_DIR=\\backup-server\dev
)

:: Backup
xcopy /s /e %DEPLOY_DIR% %BACKUP_DIR%\backup_%date%

:: Deploy
robocopy dist %DEPLOY_DIR% /MIR /MT:8

```

Automação de Testes

```

:: Suite de testes
:testSuite
echo === Iniciando Testes ===

:: Testes unitários
call npm test
if errorlevel 1 goto :testFailed

:: Testes E2E
call npm run e2e
if errorlevel 1 goto :testFailed

echo === Testes Concluídos ===
exit /b 0

:testFailed
echo === Falha nos Testes ===
exit /b 1

```

Automação de Logs

```

:: Rotação de logs
:rotateLogs
set LOG_DIR=logs
set MAX_LOGS=5

:: Remove logs antigos
forfiles /p %LOG_DIR% /m *.log /d -%MAX_LOGS% /c "cmd /c del
@path"

:: Compacta logs
for %%f in (%LOG_DIR%\*.log) do (
    7z a %LOG_DIR%\%%~nf.zip %%f
    del %%f
)

```

Integração com APIs

```
:: Chamadas REST
:checkAPI
curl -s -o response.tmp http://api.exemplo.com/status
findstr "OK" response.tmp > nul

if errorlevel 1 (
    echo API offline
    call :notifyTeam
) else (
    echo API operacional
)
```

Scripts de Manutenção

```
:: Manutenção automática
:maintenance
echo === Iniciando Manutenção ===

:: Limpeza
call :cleanTemp
call :rotateLogs
call :backupDB

:: Verificações
call :checkDiskSpace
call :checkServices
call :checkCertificates

echo === Manutenção Concluída ===
```

Troubleshooting Automático

```
:: Diagnóstico automático
:diagnose
set REPORT=diagnostic_%date%.txt

:: Coleta informações
systeminfo >> %REPORT%
tasklist >> %REPORT%
netstat -ano >> %REPORT%
dir /s /b >> %REPORT%

:: Análise
findstr "Error" *.log >> %REPORT%
```

Próximos Passos

- 1.
2. PowerShell Migration ([Introdução ao PowerShell](#))
- 3.

"Automatize o presente, controle o futuro."

Segurança no CMD: Porque Paranoia é Apenas Bom Senso

```

  _
 / _ |   _   _   _   _   _   _   _   _   _   _   _   _   _   _   _
 \_   \ / _ \ / _ \ / _ \ / _ \ / _ \ / _ \ / _ \ / _ \ / _ \ / _ \
  _   ) |   _ / ( _ | | | | | | | | | | | | | | | | | | | | | |
 | _ _ / \_ | \_ | \_ | \_ | \_ | \_ | \_ | \_ | \_ | \_ | \_ | \_ |
                                     | _ _ /

```

⚠ "Em um mundo onde até sua torradeira pode ser hackeada, seu terminal é a última linha de defesa."

A Ilusão da Segurança

Ah, você achava que estava seguro só porque usa Windows? Que fofo. Vamos mergulhar no submundo da segurança do CMD, onde cada comando é uma roleta russa digital.

Permissões: Seu Primeiro Pesadelo

```

:: Verificando permissões (ou a falta delas)
icacls "C:\Secrets"
:: Spoiler: Você provavelmente não tem acesso ao que precisa

```

Elevação de Privilégios (Ou: Como Se Tornar um Deus Digital)

```

:: O famoso "Execute como Administrador"
runas /user:Administrator cmd.exe
:: Parabéns, agora você tem poder. Use com sabedoria... ou não.

```

Criptografia para Paranoicos

```
:: Encriptando arquivos (porque você nunca sabe)
cipher /e /s:C:\ProjetosSecretos
```

```
:: Descriptografando (quando a paranoia passar)
cipher /d /s:C:\ProjetosSecretos
```

Sanitização de Dados (Ou: Como Dormir à Noite)

```
:: Limpeza segura (modo paranoia)
@echo off
set "file=segredos.txt"
for /l %%i in (1,1,7) do (
    copy /y NUL %file%
    del /f /q %file%
)
:: Se você ainda está preocupado, queime o HD
```

Firewall: Seu Amigo Neurótico

```
:: Configurando o firewall (porque sim)
netsh advfirewall firewall add rule name="Block_Evil" dir=in
action=block protocol=TCP localport=666
:: Agora você está "seguro" (use aspas mentais aqui)
```

Monitoramento de Atividades Suspeitas

```
:: Quem está te observando?
netstat -anb | findstr "ESTABLISHED"
:: Spoiler: Todo mundo está te observando
```

Log de Eventos (Ou: Seu Diário Digital Paranóico)

```
:: Verificando logs (prepare-se para pesadelos)
wevtutil qe Security /c:100 /f:text
```

:: Se você entendeu algo, parabéns, você é um unicórnio

Dicas de Sobrevivência Digital

1. **Nunca confie em scripts baixados da internet** (Mas você vai baixar mesmo assim, não é?)
2. **Sempre verifique hashes**

```
certutil -hashfile suspicious.exe SHA256  
:: Se os hashes não baterem... bem, você já estava infectado  
mesmo
```

3. **Backup é para os fracos** (Mentira, faça backup de tudo, três vezes)

Checklist do Paranoico Digital

- ☐ Firewall ativado
- ☐ Antivírus atualizado
- ☐ Permissões verificadas
- ☐ Chapéu de papel alumínio colocado
- ☐ Câmera coberta com fita
- ☐ Terminal rodando em modo seguro
- ☐ Paranoia em níveis saudáveis

Conclusão

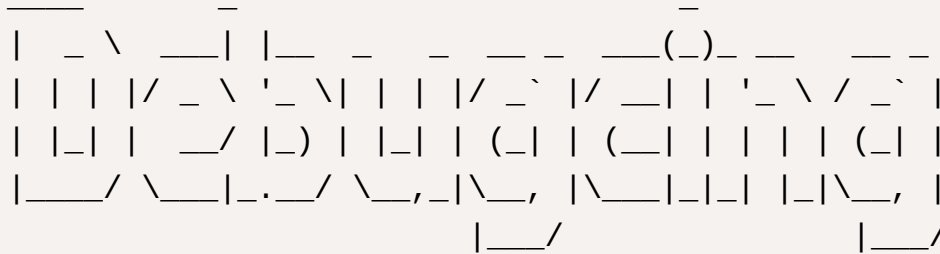
 "Se você não está paranóico com segurança, é porque não está prestando atenção."

Próximos Passos

1. PowerShell Security ([Segurança no PowerShell](#)) (Para quando o CMD não for paranoico o suficiente)
2. Advanced Security ([Técnicas Avançadas de CMD](#)) (Para os verdadeiramente perturbados)
3. Troubleshooting ([Troubleshooting Guide](#)) (Para quando tudo der errado, e vai dar)

"Em um mundo digital, paranoia não é um bug, é uma feature."

Debugging no CMD: Onde os Erros Vão Para Festa



"Se você não está tendo problemas com seu código, provavelmente não está rodando ele."

A Arte da Depuração (Ou: Como Perder Sua Sanidade com Estilo)

Debug Mode: Seu Novo Melhor Amigo

```
:: Ative o modo debug e observe o caos
@echo on
set DEBUG=true
:: Agora você pode ver tudo queimando em tempo real
```

Técnicas de Debugging Para Masoquistas Digitais

1. Echo Debug (Para os Tradicionalistas)

```
@echo off
echo [DEBUG] Iniciando script...
echo [DEBUG] Variável X = %X%
echo [DEBUG] Se você está lendo isso, algo deu errado
```

2. Logging Avançado (Porque Você Vai Precisar de Provas)

```
:: Crie um arquivo de log que ninguém vai ler
set
"LOG_FILE=%TEMP%\debug_%DATE:~-4%%DATE:~3,2%%DATE:~0,2%_%TIME:~0,2%%TIME:~3,2%.log"
echo %DATE% %TIME% - Script iniciado >> %LOG_FILE%
echo %DATE% %TIME% - Algo deu errado >> %LOG_FILE%
echo %DATE% %TIME% - Como sempre >> %LOG_FILE%
```

Ferramentas do Debugger (Seu Kit de Sobrevivência Digital)

Error Levels: Seu Guia Para o Desespero

```
:: Verificando error levels
call seu_script.bat
if errorlevel 1 (
    echo Houston, temos um problema
    goto :PANIC
)

:PANIC
echo Iniciando protocolo de pânico...
echo Considere mudar de profissão
```

Pause Estratégico (Ou: A Arte de Congelar o Tempo)

```
:: Pausa dramática para debug
set /p "DEBUG=Pressione ENTER para continuar o desastre..."
```

Técnicas Avançadas de Masoquismo

1. Step-by-Step Debugging

```
:: Debug passo a passo (para os muito pacientes)
@echo on
set "comando1=executando..."
echo %comando1%
pause
set "comando2=ainda executando..."
echo %comando2%
pause
:: Repita até perder a vontade de viver
```

2. Variável Watching (Stalking Digital)

```
:: Monitorando variáveis como um stalker
set | findstr "IMPORTANTE"
set | findstr "CRÍTICO"
:: Observe suas variáveis desaparecerem misteriosamente
```

Armadilhas Comuns (Ou: Como Não Se Enforcar com o Próprio Código)

```
:: Armadilha #1: Espaços em paths
if exist "C:\Program Files\Sua App" (
    echo Parabéns, você caiu na armadilha mais básica
)

:: Armadilha #2: Variáveis vazias
if "%VAR%"==" " (
    echo Surpresa! Sua variável evaporou
)
```

Checklist do Debugger Desesperado

- [] Echo on ativado
- [] Logs criados

- [] Sanidade mental checada
- [] Café reabastecido
- [] Stack Overflow aberto em 15 tabs
- [] Plano B de carreira atualizado

Soluções de Emergência

1. O Famoso Ctrl+C

 Quando tudo falhar, sempre existe o botão de pânico

2. Taskkill

```
:: Quando a sutileza não é uma opção  
taskkill /F /IM cmd.exe  
:: Adeus, cruel mundo dos processos
```

3. O Reboot Mágico

 Porque às vezes, o melhor debug é um reboot

Próximos Passos (Se Você Ainda Tiver Coragem)

1. Advanced Debugging ([Técnicas Avançadas de CMD](#)) (Para os verdadeiros masoquistas)
2. PowerShell Debugging ([Debugging no PowerShell](#)) (Quando o CMD não for complexo o suficiente)
3. Terapia ([Exercícios CMD: Hora de Praticar](#)) (Altamente recomendado após este módulo)

"Debug é como investigação criminal: o código é sempre culpado até que se prove o

contrário."

Exercícios CMD: Hora de Praticar

 "A prática leva à perfeição. Ou pelo menos a um histórico de comandos mais interessante."

Exercícios Básicos

Navegação (Nível: Iniciante)

1. Explorador de Diretórios

```
:: Crie esta estrutura e navegue entre as pastas
/projeto
  /src
    /components
    /utils
  /docs
  /tests
```

- ☐ Crie a estrutura usando apenas `md` e `cd`
- ☐ Navegue até `/components` usando caminho relativo
- ☐ Volte para raiz usando `cd` com caminho absoluto
- ☐ Liste todos arquivos recursivamente

2. Gerenciador de Backup

- ☐ Crie pasta `backup_YYYY_MM_DD`
- ☐ Copie todos `.txt` para backup
- ☐ Liste arquivos copiados
- ☐ Compare tamanhos originais e cópias



Exercícios Intermediários

Manipulação de Arquivos (Nível: Médio)

1. Organizador de Downloads

```
:: Crie script que:  
:: 1. Entre na pasta Downloads  
:: 2. Crie pastas por extensão  
:: 3. Mova arquivos automaticamente
```

- ☐ Identifique extensões únicas
- ☐ Crie pastas necessárias
- ☐ Mova arquivos preservando nomes
- ☐ Gere log de operações

2. Monitor de Alterações

- ☐ Monitore pasta específica
- ☐ Registre mudanças em log
- ☐ Notifique arquivos novos
- ☐ Mantenha histórico de 7 dias



Exercícios Avançados

Automação (Nível: Expert)

1. Deploy Automatizado

```
:: Crie sistema que:  
:: 1. Faça backup do atual  
:: 2. Copie novos arquivos
```

```
:: 3. Valide a cópia  
:: 4. Permita rollback
```

- ☐ Implemente verificação MD5
- ☐ Adicione log detalhado
- ☐ Crie sistema de rollback
- ☐ Notifique resultado

2. Monitor de Sistema

- ☐ Monitore CPU e memória
- ☐ Alerta uso alto (>80%)
- ☐ Registre processos ativos
- ☐ Gere relatório diário

Projetos Práticos

Projeto 1: Sistema de Backup

```
@echo off  
:: Implemente backup incremental  
:: - Compare datas de modificação  
:: - Copie apenas alterados  
:: - Mantenha estrutura  
:: - Gere relatório
```

Projeto 2: Ambiente Dev

```
@echo off  
:: Configure ambiente desenvolvimento  
:: - Clone repositório  
:: - Instale dependências
```

```
:: - Configure variáveis  
:: - Inicie serviços
```

Desafios

Desafio 1: File Finder

- Crie buscador de arquivos avançado
- Suporte wildcards e regex
- Permita filtros (data, tamanho)
- Gere relatório formatado

Desafio 2: Log Analyzer

- Processe arquivos de log
- Extraia estatísticas
- Identifique padrões
- Gere alertas

Checklist de Habilidades

Básico

- ☐ Navegação fluente
- ☐ Manipulação de arquivos
- ☐ Comandos essenciais
- ☐ Redirecionamento básico

Intermediário

- [] Scripts simples
- [] Variáveis e condições
- [] Loops básicos
- [] Processamento de texto

Avançado

- [] Funções complexas
- [] Tratamento de erros
- [] Networking
- [] Automação completa



Avaliação

Projeto Final

Crie uma ferramenta que combine:

1. Monitoramento de sistema
2. Backup automático
3. Log processing
4. Relatórios
5. Interface amigável

Critérios

- Funcionalidade
- Tratamento de erros
- Documentação

- Performance
- Manutenibilidade

Certificação

Complete todos exercícios para:

1. Domínio básico do CMD
2. Automação eficiente
3. Troubleshooting avançado
4. Integração com ferramentas

Recursos Adicionais

- Documentação Microsoft (<https://docs.microsoft.com>)
- Batch Script Tutorial (https://www.tutorialspoint.com/batch_script/)
- Fórum de Suporte (<https://stackoverflow.com>)

"A melhor forma de aprender é quebrando coisas... e depois consertando."

Integração CMD: Onde Todos os Mundos Colidem

[illegible]

⚠️ "Em um universo de containers e clouds, o CMD ainda insiste em comparecer à festa."

A Arte da Integração (Ou: Como Fazer Tecnologias Incompatíveis Dançarem Juntas)

Preparando o Terreno

```
:: Configurando o ambiente para o caos
@echo off
setlocal enabledelayedexpansion
:: Agora sim, podemos começar a quebrar coisas apropriadamente
```

Git Integration (Ou: Como Versionar Seu Desespero)

Git Básico para Sobrevivientes

```
:: Script de commit automático (use por sua conta e risco)
@echo off
set "COMMIT_MSG=Fix: Tentando consertar o que quebrei ontem"

git add .
git commit -m "%COMMIT_MSG%"
git push
```

```
if errorlevel 1 (  
    echo Parabéns, você quebrou o repositório  
    goto :PANIC  
)
```

Automação Git Avançada

```
:: Automatizando o pânico  
:gitAutomation  
git status | findstr "modified"  
if errorlevel 1 (  
    echo Milagre: Nada quebrado hoje  
) else (  
    echo Hora do commit desesperado  
    call :panicCommit  
)
```

Docker Integration (Ou: Containers de Desespero)

Docker CLI via CMD

```
:: Gerenciando containers como um hacker dos anos 90  
docker ps  
if errorlevel 1 (  
    echo Docker não está rodando... ou está?  
    echo Iniciando ritual de invocação do daemon  
    start "" "C:\Program Files\Docker\Docker\Docker Desktop.exe"  
    timeout /t 30 /nobreak  
    echo Se isso não funcionou, tente sacrificar um SSD  
)
```

Build e Deploy

```
:: Script de build que ninguém pediu  
:dockerBuild  
set "IMAGE_NAME=meu-container-do-caos"
```

```
set "TAG=latest-disaster"

docker build -t %IMAGE_NAME%:%TAG% .
if errorlevel 1 (
    echo 0 build falhou... que surpresa
    goto :PANIC
)
```

Cloud Integration (Ou: Levando Seus Problemas Para a Nuvem)

AWS CLI

```
:: Automatizando a queima de dinheiro na AWS
aws s3 sync . s3://meu-bucket-de-desastres
if errorlevel 1 (
    echo AWS Console está rindo de você agora
    echo Verificando saldo do cartão de crédito...
)
```

Azure CLI

```
:: Tentando fazer amizade com a Azure
az login
if errorlevel 1 (
    echo Microsoft não quer seu dinheiro hoje
    echo Tente novamente após reiniciar o universo
)
```

Checklist de Integração

- [] Git configurado e quebrando corretamente
- [] Docker daemon em estado questionável
- [] AWS credenciais expostas em algum lugar

- [] Azure pensando sobre a vida
- [] Cartão de crédito chorando
- [] Backup do LinkedIn atualizado

Protocolos de Emergência

1. O Protocolo de Pânico

```
:PANIC  
echo Iniciando protocolo de pânico corporativo...  
echo Enviando CV atualizado...  
echo Preparando desculpas plausíveis...
```

2. Recuperação de Desastres

```
:: Quando tudo der errado (e vai dar)  
:disasterRecovery  
echo Tentando recuperar o irrecuperável...  
echo Rezando para todos os deuses da tecnologia...
```

Próximos Passos (Se Ainda Houver Esperança)

1. Git Integration ([Git Integration com CMD: Controlando o Caos](#)) (Para masoquistas do controle de versão)
2. Docker Integration ([Docker Integration com CMD: Containers de Caos](#)) (Containerize sua dor)
3. Cloud Integration ([Cloud Integration com CMD: Problemas em Escala Global](#)) (Problemas em escala global)

"Integração é como tentar fazer gatos e cachorros viverem em harmonia... em um container Docker."

Git Integration com CMD: Controlando o Caos

$\frac{1}{\sqrt{\pi}} \int_{-\infty}^{\infty} f(x) e^{-x^2} dx = \frac{1}{\sqrt{\pi}} \int_{-\infty}^{\infty} f(x) e^{-x^2} dx$

⚠ "Git é como um checkpoint em um jogo difícil: você nunca sabe se salvou no momento certo."

Configuração Inicial (Ou: Preparando-se Para o Desastre)

Setup Básico

```
:: Configurando sua identidade (para saber quem culpar depois)
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"

:: Configurando cores (porque preto e branco é muito 1980)
git config --global color.ui auto
```

Scripts de Automação Git (Ou: Automatizando Seus Erros)

Commit Rápido

```
@echo off
:: Script para commit rápido (e arrependimento posterior)
```

```

:quickCommit
set /p COMMIT_MSG="Digite sua mensagem de commit (ou pressione
Enter para 'oops'): "
if "%COMMIT_MSG%"==" " set COMMIT_MSG=oops: provavelmente quebrei
algo

git add .
git commit -m "%COMMIT_MSG%"
git push

if errorlevel 1 (
    echo Parabéns! Você conseguiu quebrar o Git!
    goto :fixItLater
)

```

Branch Management

```

:: Criando e mudando branches (ou: como se perder em seu próprio
código)
:branchManager
set /p BRANCH_NAME="Nome da nova branch (pressione Enter para
'feature/outro-bug'): "
if "%BRANCH_NAME%"==" " set BRANCH_NAME=feature/outro-bug

git checkout -b %BRANCH_NAME%
if errorlevel 1 (
    echo Branch já existe ou você digitou algo muito criativo
    echo Tentando plano B...
    git checkout %BRANCH_NAME%
)

```

Workflows Comuns (Ou: Rituais Diários de Desespero)

Pull Before Push

```

:: Script para evitar conflitos (boa sorte com isso)
:safePush
echo Verificando se tem alguém mais quebrando o código...
git fetch

git pull
if errorlevel 1 (
    echo CONFLITO DETECTADO!
    echo Iniciando protocolo de pânico...
    goto :mergeConflictPanic
)

git push

```

Merge Conflict Resolution

```

:: Quando tudo dá errado (e vai dar)
:mergeConflictPanic
echo =====
echo PROTOCOLO DE CONFLITO DE MERGE ATIVADO
echo =====
echo 1. Não entre em pânico
echo 2. Tarde demais para o item 1
echo 3. Tentando resolver automaticamente...

git config merge.tool vimdiff
git mergetool

if errorlevel 1 (
    echo Resolução automática falhou
    echo Recomendação: Finja que não foi você
)

```

Scripts de Manutenção (Ou: Mantendo a Sanidade)

Limpeza de Branches

```
:: Removendo branches mortas (RIP)
:cleanBranches
echo Iniciando limpeza de branches...
echo (Ou: deletando evidências)

git branch --merged | findstr /v "* master main develop" >
temp.txt
for /f "tokens=*" %%a in (temp.txt) do (
    git branch -d %%a
)
del temp.txt
```

Backup de Emergência

```
:: Quando tudo mais falhar
:panicBackup
set BACKUP_DIR=backup_%date:~-4%%date:~-3,2%%date:~-0,2%
mkdir %BACKUP_DIR%
xcopy /s /e .git %BACKUP_DIR%\git\
echo Backup criado em %BACKUP_DIR%
echo (Mas você provavelmente nunca vai usar isso)
```

Protocolos de Emergência

Reset de Emergência

```
:: Botão de pânico nuclear
:nuclearOption
echo ATENÇÃO: INICIANDO PROTOCOLO DE RESET
echo Este é o botão vermelho. Tem certeza?
pause

git reset --hard HEAD~1
echo O que está feito, está feito
echo (Espero que você tenha feito backup)
```

Stash Rápido

```
:: Escondendo as evidências
:quickStash
git stash push -m "Mudanças que eu deveria ter commitado ontem"
echo Suas alterações foram escondidas
echo (Boa sorte lembrando onde)
```

Checklist de Sobrevivência Git

- [] Repositório inicializado (provavelmente do jeito errado)
- [] Remote configurado (apontando para algum lugar)
- [] Branches criadas (e perdidas)
- [] Commits feitos (e arrependidos)
- [] Conflitos resolvidos (ou ignorados)
- [] Stack Overflow aberto em várias tabs

Próximos Passos (Se Você Sobreviveu)

1. Docker Integration ([Docker Integration com CMD: Containers de Caos](#)) (Para mais aventuras em containers)
2. Cloud Integration ([Cloud Integration com CMD: Problemas em Escala Global](#)) (Leve seus problemas para a nuvem)
3. Terapia ([Exercícios CMD: Hora de Praticar](#)) (Altamente recomendado neste ponto)

"Git é como um relacionamento complicado: ninguém entende realmente como funciona, mas todo mundo finge que sim."

Docker Integration com CMD: Containers de Caos

	-	\	/	-	\	/	___			/	/	___		-	\			
									'	/		-			_)			
	_		_			_			.	\		_		-	<			
_	___	/	\	___	/	\	___		-		\	\	___		-		\	\

⚠️ "Docker é como uma caixa surpresa: você nunca sabe o que vai quebrar quando abrir."

Configuração Inicial (Ou: Preparando o Ambiente para o Desastre)

Verificação do Ambiente

```
@echo off
:: Checando se o Docker está instalado (e rezando)
docker --version
if errorlevel 1 (
    echo Docker não encontrado!
    echo Você esqueceu de instalar a fonte de todos os seus
    problemas futuros
    goto :installDocker
)
```

Status Check

```
:: Verificando se o daemon está vivo
:checkDaemon
docker info > nul 2>&1
if errorlevel 1 (
```

```
    echo O daemon Docker está morto... de novo
    echo Iniciando ritual de ressurreição
    start "" "C:\Program Files\Docker\Docker\Docker Desktop.exe"
    timeout /t 30 /nobreak
    goto :checkDaemon
)
```

Gerenciamento de Containers (Ou: Malabarismo com Caixas Virtuais)

Container Lifecycle

```
:: Script para gerenciar o ciclo de vida dos containers
:containerManager
set "CONTAINER_NAME=container-do-caos"

:: Criando um container (o que poderia dar errado?)
docker run -d --name %CONTAINER_NAME% nginx
if errorlevel 1 (
    echo Container recusou-se a iniciar
    echo Tentando convencê-lo educadamente...
    docker rm -f %CONTAINER_NAME% 2>nul
    goto :containerManager
)
```

Container Cleanup

```
:: Limpeza de containers zumbis
:zombieHunter
echo Iniciando caça aos containers mortos-vivos...
docker container prune -f
echo Se algo importante foi deletado, foi mal
```

Imagens e Builds (Ou: Arte Abstrata em Camadas)

Build Automation

```
:: Script de build com tentativas infinitas
:buildImage
set "IMAGE_NAME=app-que-funciona-local"
set "TAG=versao-otimista"

echo Tentativa de build #%ATTEMPT%
docker build -t %IMAGE_NAME%:%TAG% .
if errorlevel 1 (
    echo Build falhou... que surpresa
    set /a ATTEMPT+=1
    if %ATTEMPT% leq 3 (
        echo Tentando novamente, porque insistência é a chave
        goto :buildImage
    ) else (
        echo Desistindo... por enquanto
        goto :panicMode
    )
)
```

Image Cleanup

```
:: Liberando espaço (e deletando evidências)
:imageJanitor
echo Procurando imagens para "otimizar"...
docker image prune -a --force
echo Seu HD agradece
echo (Mas seu time de desenvolvimento não)
```

Networking (Ou: Labirinto de Conexões)

Network Creation

```
:: Criando redes Docker (mais uma camada de complexidade)
:networkCreator
docker network create rede-do-caos
```

```
if errorlevel 1 (  
    echo A rede se recusa a existir  
    echo Tentando plano B...  
    goto :useHostNetwork  
)
```

Container Connectivity

```
:: Testando conectividade (ou falta dela)  
:connectivityCheck  
docker run --rm busybox ping -c 1 google.com  
if errorlevel 1 (  
    echo Internet não encontrada  
    echo Culpando o firewall...  
)
```

Volume Management (Ou: Persistência é Superestimada)

Volume Creation

```
:: Criando volumes (porque perder dados é divertido)  
:volumeManager  
docker volume create dados-importantes  
echo Volume criado com sucesso  
echo (Boa sorte encontrando onde)
```

Backup Volumes

```
:: Script de backup (para quando der tudo errado)  
:backupVolumes  
set "BACKUP_DIR=backup_volumes_%date:~-4%%date:~3,2%%date:~0,2%"  
mkdir %BACKUP_DIR%  
  
docker run --rm -v dados-importantes:/data -v  
%cd%/BACKUP_DIR%:/backup alpine tar cvf /backup/backup.tar /data
```

```
echo Backup concluído  
echo (Espero que você não precise dele)
```

Troubleshooting (Ou: Guia de Sobrevivência)

Log Analysis

```
:: Analisando logs (ou tentando)  
:logDiver  
docker logs --tail 100 %CONTAINER_NAME%  
echo Se você entendeu esses logs, parabéns  
echo Você é oficialmente um especialista Docker
```

Container Debug

```
:: Modo debug desesperado  
:debugMode  
echo Entrando em modo de depuração...  
docker exec -it %CONTAINER_NAME% /bin/sh  
echo Se isso funcionou, foi sorte
```

Checklist de Sobrevivência Docker

- [] Docker Desktop instalado (e reiniciado 3 vezes)
- [] Containers rodando (alguns deles)
- [] Volumes montados (em algum lugar)
- [] Redes configuradas (teoricamente)
- [] Logs verificados (e ignorados)
- [] Stack Overflow como página inicial

Próximos Passos (Se Ainda Tiver Coragem)

1. Cloud Integration ([Cloud Integration com CMD: Problemas em Escala Global](#)) (Para problemas em escala global)
2. Kubernetes ([Exercícios CMD: Hora de Praticar](#)) (Porque Docker sozinho não é complexo o suficiente)
3. Terapia ([Exercícios CMD: Hora de Praticar](#)) (Seramente, você vai precisar)

"Docker é como uma cebola: tem várias camadas e provavelmente vai te fazer chorar."

Cloud Integration com CMD: Problemas em Escala Global

```
  _ _ _ _ _  
 /  _  | |  /  _  \ | | | |  _  \  
 | |  | |  | |  | |  | |  | |  | |  
 | |  _  | |  _  | |  | |  | |  | |  
 \  _  | |  \  _  /  \  _  /  | |  /
```

⚠ "Cloud é como um hotel: você paga pelo que usa, mas nunca sabe exatamente quanto vai custar no final do mês."

AWS CLI Adventures (Ou: Como Gastar Dinheiro Automaticamente)

Configuração Inicial

```
@echo off  
:: Configurando AWS CLI (e suas economias)  
aws configure  
if errorlevel 1 (  
    echo AWS não quer seu dinheiro hoje  
    echo Tente novamente após atualizar seu cartão de crédito  
    goto :panicMode  
)
```

S3 Management

```
:: Upload para S3 (ou: enviando arquivos para o void)  
:s3Upload  
set "BUCKET_NAME=bucket-do-arrependimento"  
set "LOCAL_PATH=dados-importantes"
```

```
aws s3 sync %LOCAL_PATH% s3://%BUCKET_NAME%
if errorlevel 1 (
    echo Upload falhou
    echo Mas a cobrança provavelmente passou
    goto :checkBill
)
```

Azure CLI Saga (Ou: Microsoft's Cloud Maze)

Azure Login

```
:: Tentando acessar Azure (boa sorte)
:azureLogin
az login
if errorlevel 1 (
    echo Azure está tendo um dia ruim
    echo Tente novamente em 2-3 dias úteis
    goto :useCompetitor
)
```

Resource Management

```
:: Criando recursos (e dívidas)
:createResources
set "RG_NAME=grupo-recursos-caros"
set "LOCATION=somewhere-expensive"

az group create --name %RG_NAME% --location %LOCATION%
if errorlevel 1 (
    echo Criação falhou
    echo Mas você será cobrado mesmo assim
)
```

Google Cloud Adventures (Ou: Mais Uma Nuvem Para Se Preocupar)

GCloud Setup

```
:: Configurando GCloud (porque duas clouds não são suficientes)
:gcloudInit
gcloud init
if errorlevel 1 (
    echo Google também não quer cooperar
    echo Pelo menos o Gmail ainda funciona
)
```

Container Management

```
:: Deployando containers (agora em escala cloud)
:deployContainer
set "PROJECT_ID=projeto-sem-orcamento"

gcloud run deploy --image gcr.io/%PROJECT_ID%/app
if errorlevel 1 (
    echo Deploy falhou
    echo Mas o billing continua...
)
```

Multi-Cloud Chaos (Ou: Problemas Distribuídos)

Cloud Sync

```
:: Sincronizando entre clouds (porque gostamos de sofrer)
:multiCloudSync
echo Iniciando sincronização multi-cloud...

:: AWS para Azure
aws s3 sync s3://%AWS_BUCKET% temp/
az storage blob upload-batch -d %AZURE_CONTAINER% -s temp/

:: Azure para GCloud
az storage blob download-batch -d temp/ -s %AZURE_CONTAINER%
gsutil cp -r temp/* gs://%GLOUD_BUCKET%/
```

```
echo Se algo sincronizou, foi coincidência
```

Cost Management (Ou: Arte de Queimar Dinheiro)

Budget Alerts

```
:: Configurando alertas de orçamento (tarde demais)
:budgetAlert
set "BUDGET_LIMIT=muito-dinheiro"

aws budgets create-budget --too-complicated-to-show
az monitor metrics alert create --good-luck-understanding-this
gcloud billing budgets create --why-so-complex

echo Alertas configurados
echo (Você será notificado após o estrago)
```

Disaster Recovery (Ou: Plano B, C, D...)

Backup Strategy

```
:: Script de backup multi-cloud (para quando tudo der errado)
:backupAll
echo Iniciando backup de todas as clouds...
echo (Rezando para todas as divindades da TI)

:: Backup AWS
aws s3 sync s3://%AWS_BUCKET% backup/aws/

:: Backup Azure
az storage blob download-batch -d backup/azure -s
%AZURE_CONTAINER%

:: Backup GCloud
gsutil -m cp -r gs://%GLOUD_BUCKET% backup/gcloud/
```

```
echo Backup concluído  
echo (Se você precisar usar, já é tarde demais)
```

Checklist de Sobrevivência Cloud

- ☐ Cartão de crédito com limite alto
- ☐ Contato do suporte financeiro
- ☐ Plano de recuperação de desastres
- ☐ LinkedIn atualizado
- ☐ Currículo pronto
- ☐ Passaporte em dia

Próximos Passos (Se o Orçamento Permitir)

1. Kubernetes ([Exercícios CMD: Hora de Praticar](#)) (Para quando cloud pura não é complexa o suficiente)
2. DevOps Integration ([Automação Avançada: Além dos Scripts](#)) (Automatize seus problemas)
3. Terapia Financeira ([Exercícios CMD: Hora de Praticar](#)) (Você vai precisar)

"Cloud é como um cassino: a casa sempre ganha, mas você continua jogando mesmo assim."

PowerShell: A Evolução do Terminal Windows

⚠ "PowerShell não é apenas um shell, é uma plataforma de automação."

O que é PowerShell?

PowerShell é uma plataforma de automação e shell de comando que combina:

- Shell de linha de comando
- Linguagem de scripting
- Framework de automação
- Gerenciamento de configuração

Por que PowerShell?

Vantagens sobre CMD

```
# CMD: Saída como texto  
dir | findstr ".txt"
```

```
# PowerShell: Saída como objetos  
Get-ChildItem | Where-Object { $_.Extension -eq '.txt' }
```

Recursos Exclusivos

- Pipeline orientado a objetos
- Cmdlets consistentes (Verb-Noun)
- Integração profunda com Windows
- Suporte multiplataforma (Core)

- Extensibilidade via módulos

Arquitetura Básica

Componentes Principais

```
# Verificar versão e edição
$PSVersionTable

# Listar comandos disponíveis
Get-Command

# Explorar tipos de objetos
Get-ChildItem | Get-Member
```

Pipeline & Objetos

```
# Exemplo de pipeline poderoso
Get-Process |
    Where-Object { $_.CPU -gt 100 } |
    Sort-Object CPU -Descending |
    Select-Object Name, CPU, Memory |
    Export-Csv report.csv
```

Estrutura do Curso

Fundamentos

1. Introdução ([Introdução ao PowerShell](#))

- História e evolução
- Instalação e configuração
- Conceitos básicos

2. Cmdlets Básicos ([Cmdlets: Os Comandos do PowerShell](#))

- Sintaxe e estrutura
- Comandos essenciais
- Ajuda e documentação

3. Pipeline & Objetos ([Pipeline: O Poder do Encadeamento](#))

- Conceito de pipeline
- Manipulação de objetos
- Filtragem e transformação

Intermediário

1. Scripting ([Scripting no PowerShell](#))

- Variáveis e tipos
- Funções e módulos
- Controle de fluxo

2. Módulos & Extensões ([Módulos PowerShell](#))

- Criação de módulos
- Importação e exportação
- Módulos populares

3. Tratamento de Erros ([Tratamento de Erros no PowerShell](#))

- Try/Catch
- Logging
- Debug e troubleshooting

Avançado

1. Remoting ([PowerShell Remoting](#))

- Sessões remotas
- Execução distribuída
- Segurança

2. Segurança ([Segurança no PowerShell](#))

- Políticas de execução
- Assinatura de scripts
- Boas práticas

3. DevOps & Cloud ([PowerShell DevOps Integration](#))

- Azure PowerShell
- AWS Tools
- CI/CD Integration

Ambiente de Desenvolvimento

Configuração Recomendada

```
# Profile básico
Set-ExecutionPolicy RemoteSigned
Install-Module PSReadLine
Install-Module Terminal-Icons
Install-Module PSFzf
```

Ferramentas Essenciais

- Visual Studio Code
- Windows Terminal
- PowerShell ISE
- Pester (testes)

Quick Reference

Comandos Essenciais

```
# Sistema
Get-Process      # Lista processos
Get-Service      # Lista serviços
Get-EventLog     # Logs do sistema

# Arquivos
Get-ChildItem    # Lista arquivos
Copy-Item        # Cópia
Move-Item        # Move
Remove-Item      # Remove

# Rede
Test-Connection # Ping avançado
Get-NetAdapter  # Adaptadores
Invoke-WebRequest # Requisições web
```

Melhores Práticas

Desenvolvimento

- Use verbos aprovados
- Siga convenções de nomes
- Documente com comentários
- Implemente tratamento de erros
- Escreva testes unitários

Performance

- Evite loops desnecessários

- Use pipeline adequadamente
- Otimize consultas
- Cache resultados quando possível

Integração com Ferramentas

DevOps

- Git
- Docker
- Kubernetes
- Azure DevOps
- Jenkins

Desenvolvimento

- Visual Studio
- VS Code
- Git
- Node.js
- Python

Próximos Passos

1. Introdução ao PowerShell ([Introdução ao PowerShell](#))
2. Cmdlets Fundamentais ([Cmdlets: Os Comandos do PowerShell](#))
3. Scripting Básico ([Scripting no PowerShell](#))

Recursos Adicionais

- Documentação Oficial (<https://docs.microsoft.com/powershell>)
- PowerShell Gallery (<https://www.powershellgallery.com>)
- GitHub PowerShell (<https://github.com/PowerShell/PowerShell>)
- PowerShell.org (<https://powershell.org>)

"Com grande poder vem grande capacidade de automação."

Introdução ao PowerShell

 "Bem-vindo ao PowerShell, onde tudo é um objeto e nada é apenas texto."

História e Evolução

Da Necessidade à Inovação

- 2006: Lançamento inicial (PowerShell 1.0)
- 2009: Windows PowerShell 2.0 (Remoting)
- 2012: Integração profunda com Windows
- 2016: PowerShell Core (Multiplataforma)
- 2020: PowerShell 7 (Unificação)

Instalação e Configuração

Windows 10/11

```
# Verificar versão atual
$PSVersionTable.PSVersion

# Instalar última versão
winget install Microsoft.PowerShell
```

Configuração Inicial

```
# Criar perfil pessoal
if (!(Test-Path -Path $PROFILE)) {
    New-Item -ItemType File -Path $PROFILE -Force
}
```

```
# Configurar política de execução
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Conceitos Fundamentais

1. Cmdlets

```
# Estrutura: Verbo-Substantivo
Get-Process      # Lista processos
Stop-Service     # Para serviço
New-Item         # Cria item

# Descobrir comandos
Get-Command -Verb Get
Get-Command -Noun Process
```

2. Pipeline

```
# Encadeamento de comandos
Get-Process |
    Where-Object { $_.CPU -gt 50 } |
    Sort-Object CPU -Descending |
    Select-Object Name, CPU, Memory
```

3. Objetos

```
# Tudo é objeto
$processo = Get-Process chrome
$processo.Kill() # Método
$processo.Name   # Propriedade

# Explorar objeto
$processo | Get-Member
```

Primeiros Passos

Hello, PowerShell!

```
# Saída básica
Write-Host "Hello, PowerShell!" -ForegroundColor Green

# Variáveis
$nome = "Dev"
Write-Output "Olá, $nome!"

# Input do usuário
$resposta = Read-Host "Qual seu nome?"
```

Navegação Básica

```
# Sistema de arquivos
Set-Location C:\      # cd
Get-Location          # pwd
Get-ChildItem         # ls/dir
```

Ajuda e Documentação

Sistema de Ajuda

```
# Atualizar documentação
Update-Help

# Obter ajuda
Get-Help Get-Process
Get-Help Get-Process -Examples
Get-Help Get-Process -Online
```

Comparação com CMD

Comandos Equivalentes

CMD	PowerShell	Descrição
<code>dir</code>	<code>Get-ChildItem</code>	Lista arquivos
<code>cd</code>	<code>Set-Location</code>	Muda diretório
<code>copy</code>	<code>Copy-Item</code>	Copia arquivos
<code>del</code>	<code>Remove-Item</code>	Remove itens

Ambiente de Desenvolvimento

Ferramentas Recomendadas

- Visual Studio Code + PowerShell Extension
- Windows Terminal
- PowerShell ISE (integrado)

Extensões Úteis

```
# Instalar módulos populares
Install-Module PSReadLine
Install-Module Terminal-Icons
Install-Module PSFzf
```

Dicas Práticas

Aliases e Atalhos

```
# Aliases comuns
Set-Alias -Name k -Value kubectl
Set-Alias -Name g -Value git
```

```
# Ver aliases existentes
Get-Alias | Where-Object { $_.Definition -like "*git*" }
```

Perfil Personalizado

```
# Adicionar ao $PROFILE
function prompt {
    $location = Get-Location
    "PS $location> "
}

# Funções úteis
function which($cmd) {
    Get-Command $cmd | Select-Object -ExpandProperty Source
}
```

Próximos Passos

1. Cmdlets Básicos ([Cmdlets: Os Comandos do PowerShell](#))

- Comandos essenciais
- Parâmetros comuns
- Exemplos práticos

2. Pipeline e Objetos ([Pipeline: O Poder do Encadeamento](#))

- Manipulação de dados
- Filtragem
- Transformação

3. Scripting Básico ([Scripting no PowerShell](#))

- Variáveis
- Funções

- Controle de fluxo

Recursos Adicionais

- Documentação Oficial (<https://docs.microsoft.com/powershell>)
- GitHub PowerShell (<https://github.com/PowerShell/PowerShell>)
- PowerShell Gallery (<https://www.powershellgallery.com>)

Exercícios Práticos

1. Configure seu ambiente PowerShell
2. Explore comandos básicos
3. Crie seu primeiro script
4. Experimente com pipeline
5. Personalize seu perfil

"O primeiro passo para a automação é entender suas ferramentas."

Conceitos Básicos do PowerShell

 "Entenda os fundamentos, domine o PowerShell."

Sintaxe Básica

Estrutura de Comandos

```
# Formato básico: Verbo-Substantivo
Get-Process
Set-Location
New-Item

# Com parâmetros
Get-Process -Name chrome
New-Item -Path "C:\temp" -ItemType Directory
```

Variáveis

```
# Declaração e uso
$nome = "PowerShell"
$idade = 30
$lista = @(1, 2, 3)
$hash = @{
    Chave = "Valor"
    Numero = 42
}

# Tipagem
[string]$texto = "123"
[int]$numero = 456
[datetime]$data = "2024-01-01"
```

Tipos de Dados

Tipos Básicos

```
# Strings
$texto = "PowerShell"
$texto.ToUpper()
$texto.Length

# Números
$inteiro = 42
$decimal = 3.14
$resultado = $inteiro + $decimal

# Booleanos
$verdadeiro = $true
$falso = $false
```

Arrays e Coleções

```
# Arrays
$frutas = @("maçã", "banana", "laranja")
$frutas[0]
$frutas.Count

# ArrayList
$lista = New-Object System.Collections.ArrayList
$lista.Add("item")
$lista.Remove("item")

# HashTable
$config = @{
    Servidor = "localhost"
    Porta = 8080
    Ativo = $true
}
```

Operadores

Comparação

```
# Operadores básicos
-eq  # Igual
-ne  # Diferente
-gt  # Maior que
-lt  # Menor que
-ge  # Maior ou igual
-le  # Menor ou igual

# Exemplos
$idade -gt 18
"texto" -eq "TEXT0"  # Case-insensitive por padrão
```

Lógicos

```
# AND e OR
$true -and $false
$true -or $false

# NOT
-not $true
!$false

# Combinados
($idade -gt 18) -and ($nome -eq "Admin")
```

Estruturas de Controle

Condicionais

```
# If-ElseIf-Else
if ($idade -lt 18) {
    "Menor de idade"
} elseif ($idade -eq 18) {
    "Acabou de completar 18"
} else {
```

```
    "Maior de idade"
}

# Switch
switch ($cor) {
    "vermelho" { "Pare" }
    "amarelo" { "Atenção" }
    "verde" { "Siga" }
    default { "Cor inválida" }
}
```

Loops

```
# ForEach
foreach ($item in $lista) {
    Write-Output $item
}

# For
for ($i = 0; $i -lt 10; $i++) {
    Write-Output $i
}

# While
while ($true) {
    "Loop infinito"
    break
}

# Do-While
do {
    "Executa pelo menos uma vez"
} while ($false)
```

Entrada e Saída

Input

```
# Leitura básica
$nome = Read-Host "Digite seu nome"

# Leitura segura de senha
$senha = Read-Host "Digite sua senha" -AsSecureString
```

Output

```
# Diferentes formas de saída
Write-Output "Saída normal"
Write-Host "Saída colorida" -ForegroundColor Green
Write-Warning "Aviso importante"
Write-Error "Erro crítico"
```

Manipulação de Texto

Strings

```
# Concatenação
$primeiro = "Power"
$segundo = "Shell"
$completo = $primeiro + $segundo

# Formatação
$nome = "Dev"
$mensagem = "Olá, {0}!" -f $nome

# Substituição
$texto = "Hello World"
$texto.Replace("Hello", "Hi")
```

Boas Práticas

Nomenclatura

- Use PascalCase para funções
- Use camelCase para variáveis
- Prefixe variáveis privadas com _
- Use nomes descritivos

Organização

```
# Estrutura de script
# 1. Parâmetros
param(
    [string]$Nome,
    [int]$Idade
)

# 2. Funções
function Get-UserInfo {
    param($Nome)
    # ...
}

# 3. Lógica principal
```

Exercícios Práticos

1. Crie um script que:

- Receba input do usuário
- Use diferentes tipos de dados
- Implemente estruturas de controle
- Produza output formatado

2. Manipule coleções:

- Crie e modifique arrays

- Trabalhe com hashtables
- Use loops para processamento

3. Pratique com strings:

- Concatenação
- Formatação
- Manipulação

Próximos Passos

1. Cmdlets em Detalhes ([Cmdlets: Os Comandos do PowerShell](#))
2. Pipeline Avançado ([Pipeline: O Poder do Encadeamento](#))
3. Objetos e Classes ([Objetos no PowerShell](#))

"A base sólida é o segredo do sucesso."

Cmdlets: Os Comandos do PowerShell

 "Cmdlets são as ferramentas que fazem o PowerShell poderoso."

Estrutura dos Cmdlets

Anatomia

```
# Formato: Verbo-Substantivo
Get-Process
Set-Location
New-Item

# Parâmetros comuns
-WhatIf      # Simulação
-Confirm     # Confirmação
-Verbose     # Saída detalhada
-Debug       # Informações de debug
```

Verbos Comuns

Verbo	Uso	Exemplo
Get	Obtém recursos	Get-Service
Set	Modifica recursos	Set-Content
New	Cria recursos	New-Item
Remove	Remove recursos	Remove-Item
Start	Inicia operação	Start-Process
Stop	Para operação	Stop-Service

Cmdlets Essenciais

Sistema de Arquivos

```
# Navegação
Set-Location C:\      # Muda diretório
Get-Location          # Mostra diretório atual
Push-Location         # Salva localização atual
Pop-Location          # Retorna à localização salva

# Manipulação de arquivos
Get-ChildItem          # Lista conteúdo
New-Item -Type File    # Cria arquivo
Copy-Item              # Copia
Move-Item              # Move
Remove-Item            # Remove
```

Processos e Serviços

```
# Processos
Get-Process           # Lista processos
```

```
Start-Process      # Inicia processo
Stop-Process      # Para processo
Wait-Process      # Aguarda processo

# Serviços
Get-Service       # Lista serviços
Start-Service     # Inicia serviço
Stop-Service      # Para serviço
Restart-Service   # Reinicia serviço
```

Rede

```
# Conectividade
Test-Connection   # Ping avançado
Test-NetConnection # Teste TCP
Get-NetAdapter    # Info adaptadores
Get-NetIPAddress  # Endereços IP

# Web
Invoke-WebRequest # Requisição web
Invoke-RestMethod  # API REST
```

Sistema

```
# Informações
Get-ComputerInfo # Info do sistema
Get-Disk         # Info de discos
Get-EventLog     # Logs do sistema
Get-HotFix       # Updates instalados

# Gerenciamento
Restart-Computer # Reinicia
Stop-Computer    # Desliga
Clear-Host       # Limpa tela
```

Parâmetros Avançados

Parâmetros Comuns

```
# Controle de execução
-WhatIf          # Simula execução
-Confirm         # Pede confirmação
-Verbose         # Saída detalhada
-Debug           # Modo debug
-ErrorAction     # Tratamento de erro
```

Pipeline Input

```
# Aceita input do pipeline
Get-Process | Stop-Process
Get-ChildItem *.txt | Remove-Item

# Filtragem
Get-Service | Where-Object Status -eq 'Running'
Get-Process | Sort-Object CPU -Descending
```

Descoberta e Ajuda

Encontrar Cmdlets

```
# Busca por verbo
Get-Command -Verb Get
Get-Command -Verb Set

# Busca por substantivo
Get-Command -Noun Process
Get-Command -Noun Item

# Busca por módulo
Get-Command -Module Microsoft.PowerShell.Security
```

Sistema de Ajuda

```
# Ajuda básica
Get-Help Get-Process
Get-Help Get-Service -Detailed
Get-Help New-Item -Full
Get-Help About_Variables

# Exemplos
Get-Help Get-Process -Examples
```

Módulos e Extensões

Gerenciamento de Módulos

```
# Listar e instalar
Get-Module -ListAvailable
Install-Module -Name PSReadLine
Update-Module -Name PSReadLine
Remove-Module -Name PSReadLine

# Importar e exportar
Import-Module Az
Export-ModuleMember -Function Get-Something
```

Boas Práticas

Segurança

```
# Verificar antes de executar
Get-Process -Name "notepad" | Stop-Process -WhatIf

# Confirmar ações perigosas
Remove-Item -Path "C:\Important" -Confirm
```

```
# Tratar erros
Get-Process -ErrorAction SilentlyContinue
```

Performance

```
# Otimizar pipeline
Get-Process |
    Where-Object {$_.CPU -gt 50} |
    Sort-Object CPU -Descending |
    Select-Object -First 5

# Usar parâmetros específicos
Get-Process -Name "chrome" # Melhor que filtrar depois
```

Exercícios Práticos

1. Gerenciamento de Arquivos

- Liste todos arquivos .txt recursivamente
- Copie arquivos com confirmação
- Mova arquivos usando pipeline

2. Processos e Serviços

- Liste processos por uso de CPU
- Gerencie serviços específicos
- Use WhatIf para simular ações

3. Descoberta

- Encontre cmdlets por verbo
- Explore ajuda e exemplos
- Pratique com parâmetros comuns

Próximos Passos

1. Pipeline Avançado ([Pipeline: O Poder do Encadeamento](#))
2. Objetos PowerShell ([Objetos no PowerShell](#))
3. Scripting ([Scripting no PowerShell](#))

"Um bom artesão conhece suas ferramentas."

Pipeline: O Poder do Encadeamento

 "O pipeline é o que torna o PowerShell verdadeiramente poderoso."

Conceitos Fundamentais

O que é Pipeline?

```
# Estrutura básica
Comando1 | Comando2 | Comando3

# Exemplo prático
Get-Process | Sort-Object CPU -Descending | Select-Object -First 5
```

Fluxo de Objetos

```
# Cada comando passa objetos, não texto
Get-Service |
    Where-Object Status -eq 'Running' |
    Select-Object Name, Status

# Verificar tipo de objeto
Get-Process | Get-Member
```

Operadores de Pipeline

Pipeline Básico

```
# Operador |
Get-ChildItem | Sort-Object Length
```

```
Get-Process | Export-Csv processos.csv  
Dir | Where-Object Length -gt 1MB
```

Pipeline com Atribuição

```
# Operador |>  
$processos = Get-Process |  
    Sort-Object CPU -Descending |  
    Select-Object -First 10
```

Cmdlets de Pipeline

Filtragem

```
# Where-Object  
Get-Service | Where-Object {$_.Status -eq 'Running'}  
Get-Process | Where-Object CPU -gt 10  
  
# Select-Object  
Get-Process |  
    Select-Object Name, CPU, WorkingSet |  
    Where-Object CPU -gt 10
```

Ordenação

```
# Sort-Object  
Get-ChildItem |  
    Sort-Object Length -Descending  
  
# Group-Object  
Get-Process |  
    Group-Object Company |  
    Sort-Object Count -Descending
```

Transformação

```
# ForEach-Object
Get-Process |
    ForEach-Object {
        $_.Name + " uses " + [math]::Round($_.CPU,2) + "% CPU"
    }

# Select-Object com expressões
Get-Process |
    Select-Object Name,
        @{N='Memory(MB)';E={$_.WorkingSet/1MB}},
        @{N='CPU%';E={$_.CPU}}
```

Técnicas Avançadas

Pipeline com Funções

```
# Função que aceita pipeline
function Convert-ToMB {
    param(
        [Parameter(ValueFromPipeline=$true)]
        $bytes
    )
    process {
        $mb = $bytes/1MB
        [math]::Round($mb, 2)
    }
}

# Uso
Get-ChildItem |
    Select-Object Name,
        @{N='Size(MB)';E={$_.Length | Convert-ToMB}}
```

Processamento em Lote

```
# Begin, Process, End
function Process-Items {
    begin {
        $total = 0
    }
    process {
        $total += $_.Length
    }
    end {
        "Total: $($total/1MB) MB"
    }
}
```

```
Get-ChildItem | Process-Items
```

Otimização

Performance

```
# Evitar pipeline desnecessário
# Ruim
Get-Process | Where-Object {$_.Name -eq "chrome"}

# Melhor
Get-Process -Name "chrome"

# Pipeline eficiente
Get-Process |
    Where-Object CPU -gt 10 |
    Sort-Object CPU -Descending |
    Select-Object -First 5
```

Memória

```
# Streaming vs Coleção
# Alto uso de memória
```

```
$todos = Get-ChildItem -Recurse
$todos | Where-Object {$_.Length -gt 1MB}

# Streaming eficiente
Get-ChildItem -Recurse |
    Where-Object {$_.Length -gt 1MB}
```

Depuração

Debug de Pipeline

```
# Tee-Object para debug
Get-Process |
    Tee-Object -Variable processos |
    Where-Object CPU -gt 10

# Verbose output
Get-Process -Verbose |
    Where-Object {$_.CPU -gt 10} -Verbose
```

Tratamento de Erros

```
# ErrorAction no pipeline
Get-Process |
    Stop-Process -WhatIf -ErrorAction SilentlyContinue

# Try/Catch com pipeline
try {
    Get-Process |
        Where-Object {$_.CPU -gt 10} |
        Stop-Process -ErrorAction Stop
} catch {
    Write-Warning "Erro ao parar processos: $_"
}
```

Exercícios Práticos

1. Manipulação de Arquivos

```
# Liste arquivos grandes
Get-ChildItem -Recurse |
    Where-Object Length -gt 100MB |
    Select-Object FullName, @{N='Size(MB)';E={$_.Length/1MB}}
```

2. Análise de Processos

```
# Top consumidores de memória
Get-Process |
    Sort-Object WorkingSet -Descending |
    Select-Object -First 10 |
    Format-Table Name,
        @{N='Memory(MB)';E=
    {[math]::Round($_.WorkingSet/1MB,2)}},
    CPU
```

3. Relatório de Serviços

```
# Status dos serviços
Get-Service |
    Group-Object Status |
    Select-Object Name, Count |
    Sort-Object Count -Descending
```

Próximos Passos

1. Objetos PowerShell ([Objetos no PowerShell](#))
2. Scripting Avançado ([Scripting no PowerShell](#))
3. Módulos e Funções ([Módulos PowerShell](#))

"O pipeline é como LEGO: pequenas peças se conectam para criar algo maior."

Objetos no PowerShell

 "No PowerShell, tudo é um objeto - essa é a chave para sua potência."

Fundamentos de Objetos

O que são Objetos?

```
# Objetos têm propriedades e métodos
$processo = Get-Process chrome
$processo.Name          # Propriedade
$processo.Kill()        # Método

# Tipos de objetos
$processo.GetType()
[System.Diagnostics.Process]
```

Explorando Objetos

```
# Get-Member
Get-Process | Get-Member
Get-Service | Get-Member -MemberType Property
Get-ChildItem | Get-Member -MemberType Method

# Tipos específicos
$data = Get-Date
$data | Get-Member
```

Propriedades e Métodos

Trabalhando com Propriedades

```
# Acessando propriedades
$arquivo = Get-Item ".\exemplo.txt"
$arquivo.Name
```

```
$arquivo.Length
$arquivo.LastWriteTime

# Propriedades calculadas
$arquivo | Select-Object Name,
    @{N='SizeKB';E={$_.Length/1KB}},
    @{N='Age';E={(Get-Date) - $_.CreationTime}}
```

Usando Métodos

```
# Métodos comuns
$texto = "PowerShell"
$texto.ToUpper()
$texto.Replace("Power", "Super")

# Métodos com parâmetros
$data = Get-Date
$data.AddDays(7)
$data.ToString("yyyy-MM-dd")
```

Criando Objetos

Objetos Personalizados

```
# PSCustomObject
$usuario = [PSCustomObject]@{
    Nome = "João"
    Idade = 30
    Ativo = $true
}

# Adicionando propriedades
$usuario | Add-Member -MemberType NoteProperty -Name "Email" -
Value "joao@email.com"
```

Classes Personalizadas

```
# Definindo uma classe
class Pessoa {
    [string]$Nome
    [int]$Idade

    Pessoa([string]$nome, [int]$idade) {
        $this.Nome = $nome
        $this.Idade = $idade
    }

    [string]ToString() {
        return "$($this.Nome) ($($this.Idade) anos)"
    }
}

# Usando a classe
$ Pessoa = [Pessoa]::new("Maria", 25)
```

Manipulação de Objetos

Filtragem e Seleção

```
# Where-Object
Get-Process |
    Where-Object {$_.CPU -gt 10} |
    Select-Object Name, CPU, WorkingSet

# Select-Object avançado
Get-Service |
    Select-Object Name, Status,
        @{N='RunTime';E={(Get-Date) - $_.StartTime}}
```

Transformação

```
# ForEach-Object
Get-Process |
```

```
ForEach-Object {  
    [PSCustomObject]@{  
        Nome = $_.Name  
        MemoriaMB = [math]::Round($_.WorkingSet/1MB, 2)  
        CPU = $_.CPU  
    }  
}
```

Coleções de Objetos

Arrays

```
# Array de objetos  
$processos = @(Get-Process)  
$processos[0]  
$processos.Count  
  
# Filtragem de array  
$processos.Where({$_ .CPU -gt 10})
```

ArrayList

```
# Criando ArrayList  
$lista = New-Object System.Collections.ArrayList  
$lista.Add([PSCustomObject]@{  
    Nome = "Item 1"  
    Valor = 100  
})  
  
# Manipulando  
$lista.RemoveAt(0)  
$lista.Clear()
```

Hashtables

```
# Hashtable como objeto
$config = @{
    Servidor = "localhost"
    Porta = 8080
    SSL = $true
}

# Convertendo para PSObject
$configObj = [PSCustomObject]$config
```

Tipos Avançados

Tipos Genéricos

```
# List<T>
$lista = New-Object System.Collections.Generic.List[string]
$lista.Add("PowerShell")
$lista.AddRange(@("Objetos", "Pipeline"))

# Dictionary<K,V>
$dict = New-Object
'System.Collections.Generic.Dictionary[string,int]'
$dict.Add("Um", 1)
$dict.Add("Dois", 2)
```

Conversão de Tipos

```
# Conversão explícita
[int]"42"
[datetime]"2024-01-01"
[bool]1

# Conversão de objetos
$json = $objeto | ConvertTo-Json
$xml = $objeto | ConvertTo-Xml
```

Boas Práticas

Tipagem Forte

```
# Declaração com tipo
[string]$nome = "PowerShell"
[int]$idade = 30
[datetime]$data = Get-Date

# Parâmetros tipados
function Get-Usuario {
    param(
        [string]$nome,
        [int]$idade
    )
    # ...
}
```

Validação

```
# Validação de propriedades
class Usuario {
    [ValidateNotNullOrEmpty()]
    [string]$Nome

    [ValidateRange(0,120)]
    [int]$Idade

    [ValidatePattern("^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$")]
    [string]$Email
}
```

Exercícios Práticos

1. Criação de Objetos

```
# Crie uma classe Livro
class Livro {
    [string]$Titulo
    [string]$Autor
    [int]$Ano
    [double]$Preco

    [string]ToString() {
        return "$($this.Titulo) por $($this.Autor)"
    }
}
```

2. Manipulação de Coleções

```
# Crie uma biblioteca
$biblioteca = [System.Collections.Generic.List[Livro]]::new()
$biblioteca.Add([Livro]@{
    Titulo = "PowerShell in Action"
    Autor = "Bruce Payette"
    Ano = 2017
    Preco = 59.99
})
```

3. Transformação de Dados

```
# Relatório de livros
$biblioteca |
    Select-Object Titulo, Autor,
        @{N='PrecoFormatado';E={"R$ " + $_.Preco}},
        @{N='Idade';E={(Get-Date).Year - $_.Ano}}
```

Próximos Passos

1. Pipeline Avançado ([Pipeline: O Poder do Encadeamento](#))
2. Scripting com Objetos ([Scripting no PowerShell](#))

3. Módulos e Classes ([Módulos PowerShell](#))

"Objetos são a linguagem do PowerShell - aprenda a falar fluentemente."

PowerShell Avançado

 "Domine os recursos avançados para elevar seu domínio do PowerShell."

Scripting Avançado

Módulos e Funções

```
# Estrutura de módulo
New-ModuleManifest -Path .\MyModule.psd1 -ModuleVersion "1.0" `
    -Author "Seu Nome" -Description "Descrição do módulo"

# Funções avançadas
function Get-SystemInfo {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$ComputerName,

        [switch]$IncludeNetwork
    )

    begin {
        Write-Verbose "Iniciando coleta de informações..."
    }

    process {
        # Lógica principal
    }

    end {
        Write-Verbose "Coleta finalizada."
    }
}
```

Programação Orientada a Objetos

Classes e Tipos

```
# Definição de classe
class ServerConfig {
    [string]$Hostname
    [int]$Port
    [bool]$SSL

    # Construtor
    ServerConfig([string]$host, [int]$port) {
        $this.Hostname = $host
        $this.Port = $port
        $this.SSL = $false
    }

    # Método
    [bool]Test() {
        # Lógica de teste
        return $true
    }
}
```

Manipulação Avançada de Dados

Expressões Regulares

```
# Regex avançado
$pattern = '(?<protocol>https?):\\/(?<domain>[\\w\\.-]+)(?<path>\\/.*)?'
$url = "https://github.com/powershell"

if ($url -match $pattern) {
    $protocol = $matches['protocol']
    $domain = $matches['domain']
}
```

```
$path = $matches['path']  
}
```

XML e JSON

```
# Manipulação XML  
[xml]$config = Get-Content .\config.xml  
$config.SelectNodes("//server[@enabled='true']")  
  
# JSON avançado  
$json = @{  
    servers = @(  
        @{  
            name = "prod"  
            ip = "10.0.0.1"  
        }  
    )  
} | ConvertTo-Json -Depth 10
```

Automação Avançada

Jobs e Workflows

```
# Jobs em background  
Start-Job -ScriptBlock {  
    Get-Process | Where-Object CPU -gt 50  
} -Name "MonitorCPU"  
  
# Workflow  
workflow Backup-Infrastructure {  
    parallel {  
        InlineScript { Backup-Database }  
        InlineScript { Backup-FileSystem }  
    }  
}
```

Remoting Avançado

```
# Sessões persistentes
$session = New-PSSession -ComputerName "server01"
Invoke-Command -Session $session -ScriptBlock {
    # Comandos remotos
}

# Multi-threading
$servers = "server01", "server02", "server03"
$servers | ForEach-Object -ThrottleLimit 10 -Parallel {
    Get-Service -ComputerName $_
}
```

Segurança Avançada

Criptografia

```
# Certificados
$cert = Get-PfxCertificate -FilePath .\cert.pfx
$encrypted = $cert.PublicKey.Key.Encrypt($bytes, $true)

# Secure String
$secureString = ConvertTo-SecureString "senha" -AsPlainText -Force
$encrypted = ConvertFrom-SecureString $secureString
```

JEA (Just Enough Administration)

```
# Configuração JEA
New-PSSessionConfigurationFile -Path .\JEAConfig.pssc `
    -SessionType RestrictedRemoteServer `
    -VisibleCmdlets "Get-Process", "Stop-Process" `
    -VisibleFunctions "Get-SystemInfo"
```

Debugging Avançado

Técnicas de Debug

```
# Pontos de interrupção
Set-PSBreakpoint -Script .\script.ps1 -Line 10
Set-PSBreakpoint -Command "Get-Process" -Action {
    Write-Host "Breakpoint atingido!"
}

# Trace
Trace-Command -Name ParameterBinding -Expression {
    Get-Process
} -PSHost
```

Otimização e Performance

Medição de Performance

```
# Benchmark
Measure-Command {
    1..1000 | ForEach-Object { Get-Random }
}

# Profile
$timer = [System.Diagnostics.Stopwatch]::StartNew()
# código a medir
$timer.Stop()
$timer.Elapsed
```

Boas Práticas

```
# Pipeline eficiente
$processes = Get-Process # Melhor que repetir chamadas
$processes | Where-Object { $_.CPU -gt 50 }

# Uso de memória
Get-ChildItem -Recurse | ForEach-Object {
```

```
# Processamento em streaming
} # Melhor que carregar tudo na memória
```

Integração com APIs

REST e Web Services

```
# Chamadas REST
$headers = @{
    'Authorization' = "Bearer $token"
    'Content-Type' = 'application/json'
}

$response = Invoke-RestMethod -Uri $apiUrl `
    -Method Post `
    -Headers $headers `
    -Body ($body | ConvertTo-Json)
```

Exercícios Avançados

1. Automação Completa

```
# Crie um script que:
# - Monitora recursos do sistema
# - Envia alertas por email
# - Registra logs
# - Executa ações corretivas
```

2. Framework de Testes

```
# Implemente testes unitários
Describe "Módulo de Sistema" {
    Context "Função Get-SystemInfo" {
        It "Retorna objeto válido" {
            $result = Get-SystemInfo
            $result | Should -Not -BeNullOrEmpty
        }
    }
}
```

```
}  
}  
}
```

3. Módulo Profissional

```
# Desenvolva um módulo completo com:  
# - Manifesto  
# - Funções avançadas  
# - Documentação  
# - Testes  
# - CI/CD
```

Próximos Passos

1. Scripting Avançado ([Scripting no PowerShell](#))
2. Módulos e Extensões ([Módulos PowerShell](#))
3. DevOps Integration ([PowerShell DevOps Integration](#))

"O verdadeiro poder do PowerShell está nos detalhes avançados."

Scripting no PowerShell

 "Scripts são a arte de automatizar o impossível."

Fundamentos de Scripting

Estrutura Básica

```
# Cabeçalho do script
<#
.SYNOPSIS
    Descrição breve do script
.DESCRIPTION
    Descrição detalhada do script
.PARAMETER Nome
    Descrição do parâmetro
.EXAMPLE
    .\Script.ps1 -Nome "Exemplo"
#>
param(
    [string]$Nome,
    [int]$Idade = 30
)

# Configurações iniciais
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
```

Controle de Fluxo

Condicionais

```
# If-ElseIf-Else
if ($valor -gt 100) {
    "Valor alto"
```

```
} elseif ($valor -gt 50) {  
    "Valor médio"  
} else {  
    "Valor baixo"  
}  
  
# Switch  
switch ($opcao) {  
    "A" { "Opção A selecionada" }  
    "B" { "Opção B selecionada" }  
    default { "Opção inválida" }  
}
```

Loops

```
# ForEach  
foreach ($item in $colecao) {  
    $item.ProcessarAlgo()  
}  
  
# While  
while ($condicao) {  
    # Processamento  
}  
  
# Do-While  
do {  
    # Executa pelo menos uma vez  
} while ($condicao)  
  
# For  
for ($i = 0; $i -lt 10; $i++) {  
    # Loop tradicional  
}
```

Funções

Funções Avançadas

```
function Get-UserInfo {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory, Position=0)]
        [string]$Username,

        [Parameter()]
        [switch]$Detailed
    )

    begin {
        Write-Verbose "Iniciando busca..."
    }

    process {
        # Lógica principal
    }

    end {
        Write-Verbose "Busca finalizada."
    }
}
```

Parâmetros Avançados

```
function Test-Parameters {
    param(
        [ValidateNotNullOrEmpty()]
        [string]$Nome,

        [ValidateRange(0,120)]
        [int]$Idade,

        [ValidateSet("A", "B", "C")]
        [string]$Opcao,
```

```

        [ValidateScript({Test-Path $_})]
        [string]$Arquivo
    )
}

```

Tratamento de Erros

Try-Catch

```

try {
    # Código que pode gerar erro
    $resultado = 10 / $divisor
} catch [System.DivideByZeroException] {
    Write-Error "Divisão por zero!"
} catch {
    Write-Error $_.Exception.Message
} finally {
    # Sempre executa
    Write-Host "Operação finalizada"
}

```

Logging

```

function Write-Log {
    param(
        [string]$Message,
        [string]$Level = "INFO"
    )

    $logLine = "$(Get-Date -Format 'yyyy-MM-dd
HH:mm:ss')|$Level|$Message"
    Add-Content -Path ".\app.log" -Value $logLine
}

```

Módulos

Criação de Módulos

```
# Module Manifest
New-ModuleManifest -Path .\MyModule.psd1 `
    -ModuleVersion "1.0" `
    -Author "Seu Nome" `
    -Description "Descrição do módulo" `
    -FunctionsToExport @('Get-Something', 'Set-Something')

# Module Script
$functions = @(
    'Public\*.ps1'
    'Private\*.ps1'
)

foreach ($function in $functions) {
    . $PSScriptRoot\$function
}
```

Boas Práticas

Organização

```
# Estrutura de diretórios
MyModule/
├── Public/
│   └── Export-Function.ps1
├── Private/
│   └── Helper-Function.ps1
├── Tests/
│   └── Module.Tests.ps1
├── MyModule.psd1
└── MyModule.psm1
```

Documentação

```
function Get-Something {  
    <#  
    .SYNOPSIS  
        Breve descrição  
    .DESCRIPTION  
        Descrição detalhada  
    .PARAMETER Name  
        O que é este parâmetro  
    .EXAMPLE  
        Get-Something -Name "Test"  
        Exemplo de uso  
    .NOTES  
        Informações adicionais  
    #>  
    param($Name)  
}
```

Testes

Pester Tests

```
Describe "Função Get-Something" {  
    Context "Parâmetros válidos" {  
        It "Retorna resultado esperado" {  
            $result = Get-Something -Name "Test"  
            $result | Should -Not -BeNullOrEmpty  
        }  
  
        It "Gera erro com input inválido" {  
            { Get-Something -Name "" } |  
                Should -Throw  
        }  
    }  
}
```

Debugging

Técnicas de Debug

```
# Breakpoints
Set-PSBreakpoint -Script .\script.ps1 -Line 10
Set-PSBreakpoint -Variable $varName -Mode Write

# Debug output
Write-Debug "Valor: $valor"
Write-Verbose "Processando item: $item"
```

Exercícios Práticos

1. Script de Automação

```
# Crie um script que:
# - Aceita parâmetros
# - Valida input
# - Trata erros
# - Gera logs
# - Produz output formatado
```

2. Módulo Utilitário

```
# Desenvolva um módulo com:
# - Funções públicas e privadas
# - Documentação completa
# - Testes unitários
# - Manifesto
```

3. Script de Monitoramento

```
# Implemente monitoramento que:
# - Coleta métricas do sistema
# - Processa dados
```

```
# - Gera alertas  
# - Executa ações corretivas
```

Próximos Passos

1. PowerShell Avançado ([PowerShell Avançado](#))
2. Módulos e Extensões ([Módulos PowerShell](#))
3. Integração DevOps ([PowerShell DevOps Integration](#))

"Scripts são a ponte entre ideias e automação."

Módulos PowerShell

⚠ "Módulos são a base da reutilização e distribuição de código no PowerShell."

Fundamentos de Módulos

Estrutura Básica

```
# Estrutura recomendada de módulo
MyModule/
├── Public/           # Funções exportadas
├── Private/          # Funções internas
├── Tests/            # Testes unitários
├── MyModule.psd1     # Manifesto
└── MyModule.psm1     # Script do módulo
```

Criação de Módulos

Manifesto do Módulo

```
New-ModuleManifest -Path .\MyModule.psd1 `
  -ModuleVersion "1.0.0" `
  -Author "Seu Nome" `
  -Description "Descrição do módulo" `
  -FunctionsToExport @('Get-Something', 'Set-Something')
```

Script do Módulo

```
# MyModule.psm1
$Public = @( Get-ChildItem -Path $PSScriptRoot\Public\*.ps1 -
Recurse )
$Private = @( Get-ChildItem -Path $PSScriptRoot\Private\*.ps1 -
Recurse )

foreach($import in @($Public + $Private)) {
```

```
try {
    . $import.FullName
} catch {
    Write-Error "Falha ao importar função $($import.FullName):
$_"
}
}

Export-ModuleMember -Function $Public.BaseName
```

Gerenciamento de Módulos

Comandos Essenciais

```
# Listar módulos
Get-Module -ListAvailable
Get-Module # módulos carregados

# Instalar e atualizar
Install-Module -Name PSReadLine
Update-Module -Name PSReadLine

# Importar e remover
Import-Module MyModule
Remove-Module MyModule
```

Publicação

PowerShell Gallery

```
# Publicar módulo
Publish-Module -Name MyModule -NuGetApiKey $key

# Configurar repositório
Register-PSRepository -Name LocalRepo `
```

```
-SourceLocation "\\server\share" `
-InstallationPolicy Trusted
```

Boas Práticas

Organização

- Separe funções públicas e privadas
- Use versionamento semântico
- Documente todas as funções
- Implemente testes unitários
- Mantenha compatibilidade

Documentação

```
function Get-Something {
    <#
        .SYNOPSIS
            Breve descrição
        .DESCRIPTION
            Descrição detalhada
        .PARAMETER Name
            Descrição do parâmetro
        .EXAMPLE
            Get-Something -Name "Test"
    #>
    param($Name)
}
```

Exercícios Práticos

1. Módulo Básico

```
# Crie um módulo simples com:  
# - 2 funções públicas  
# - 1 função privada  
# - Manifesto completo
```

2. Módulo Avançado

```
# Desenvolva um módulo com:  
# - Pipeline support  
# - Error handling  
# - Logging  
# - Testes Pester
```

Próximos Passos

1. Error Handling ([Tratamento de Erros no PowerShell](#))
2. Advanced PowerShell ([PowerShell Avançado](#))
3. DevOps Integration ([PowerShell DevOps Integration](#))

"Bons módulos são a fundação de código sustentável."

Tratamento de Erros no PowerShell

 "Erros são inevitáveis, tratá-los adequadamente é essencial."

Fundamentos

Tipos de Erros

- Erros de terminação (Terminating)
- Erros de não-terminação (Non-terminating)
- Exceções gerenciadas (.NET)

ErrorActionPreference

```
# Configurações globais
$ErrorActionPreference = 'Stop'           # Para em qualquer erro
$ErrorActionPreference = 'Continue'       # Continua após erro
$ErrorActionPreference = 'SilentlyContinue' # Ignora erros
$ErrorActionPreference = 'Inquire'        # Pergunta o que fazer
```

Try-Catch-Finally

Estrutura Básica

```
try {
    # Código que pode gerar erro
    Get-Process -Name "NonExistentProcess"
} catch {
    # Tratamento do erro
    Write-Error "Processo não encontrado: $_"
} finally {
    # Sempre executa
```

```
Write-Host "Operação finalizada"
}
```

Tipos Específicos

```
try {
    throw [System.IO.FileNotFoundException]::new(
        "Arquivo não encontrado"
    )
} catch [System.IO.FileNotFoundException] {
    Write-Error "Arquivo não existe"
} catch [System.Exception] {
    Write-Error "Erro genérico: $_"
}
```

Logging

Sistema de Log

```
function Write-Log {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$Message,

        [ValidateSet('INFO','WARN','ERROR')]
        [string]$Level = 'INFO',

        [string]$LogPath = ".\app.log"
    )

    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $logLine = "$timestamp [$Level] $Message"
    Add-Content -Path $LogPath -Value $logLine

    if ($Level -eq 'ERROR') {
```

```
        Write-Error $Message
    }
}
```

Debug e Troubleshooting

Ferramentas de Debug

```
# Break points
Set-PSBreakpoint -Script .\script.ps1 -Line 10
Set-PSBreakpoint -Command Get-Process

# Trace
Trace-Command -Name ParameterBinding -Expression {
    Get-Process
} -PSHost
```

Verbose Output

```
function Get-SystemInfo {
    [CmdletBinding()]
    param()

    Write-Verbose "Coletando informações..."
    Get-ComputerInfo
    Write-Verbose "Coleta finalizada"
}

Get-SystemInfo -Verbose
```

Boas Práticas

Validação de Entrada

```
function Set-UserProfile {
    [CmdletBinding()]
```

```

    param(
        [Parameter(Mandatory)]
        [ValidateNotNullOrEmpty()]
        [string]$Username,

        [ValidateRange(0,120)]
        [int]$Age,

        [ValidateScript({Test-Path $_})]
        [string]$ProfilePath
    )
}

```

Tratamento Robusto

```

function Invoke-RiskyOperation {
    [CmdletBinding()]
    param($Path)

    begin {
        Write-Log "Iniciando operação" -Level INFO
    }

    process {
        try {
            # Operação principal
            Write-Verbose "Processando $Path"

            if (-not (Test-Path $Path)) {
                throw "Caminho não existe: $Path"
            }

            # Mais operações...

        } catch {
            Write-Log $_.Exception.Message -Level ERROR
            throw
        }
    }
}

```

```
    }  
  }  
  
  end {  
    Write-Log "Operação finalizada" -Level INFO  
  }  
}
```

Exercícios Práticos

1. Sistema de Log

```
# Implemente um sistema de log que:  
# - Registra em arquivo  
# - Suporta níveis diferentes  
# - Inclui stack trace
```

2. Tratamento Avançado

```
# Crie uma função que:  
# - Valida entradas  
# - Trata múltiplos erros  
# - Usa logging  
# - Limpa recursos
```

Próximos Passos

1. Advanced PowerShell ([PowerShell Avançado](#))
2. Debugging ([Debugging no PowerShell](#))
3. Best Practices ([Melhores Práticas](#))

"Erros bem tratados são oportunidades de melhoria."

PowerShell Remoting

 "Gerencie qualquer servidor Windows, de qualquer lugar."

Fundamentos do Remoting

Configuração Básica

```
# Habilitar remoting no servidor
Enable-PSRemoting -Force

# Configurar TrustedHosts no cliente
Set-Item WSMan:\localhost\Client\TrustedHosts -Value "*" -Force

# Verificar configuração
Test-WSMan
Get-PSSessionConfiguration
```

Sessões Remotas

Conexões Básicas

```
# Sessão interativa
Enter-PSSession -ComputerName "servidor01"

# Comando único
Invoke-Command -ComputerName "servidor01" -ScriptBlock {
    Get-Process
}

# Múltiplos servidores
Invoke-Command -ComputerName "srv01","srv02" -ScriptBlock {
    Get-Service | Where-Object Status -eq "Running"
}
```

Sessões Persistentes

```
# Criar sessão
$session = New-PSSession -ComputerName "servidor01"

# Usar sessão
Invoke-Command -Session $session -ScriptBlock {
    $processes = Get-Process
    $services = Get-Service
    return @{
        Processes = $processes
        Services = $services
    }
}

# Fechar sessão
Remove-PSSession $session
```

Autenticação e Segurança

Credenciais

```
# Criar credencial
$cred = Get-Credential

# Usar credencial
Enter-PSSession -ComputerName "servidor01" -Credential $cred

# Certificados
Enter-PSSession -ComputerName "servidor01" `
    -CertificateThumbprint "ABC123..." `
    -UseSSL
```

Configurações de Segurança

```
# Configurar HTTPS
Set-WSManQuickConfig -UseSSL
```

```
# Configurar endpoints
Register-PSSessionConfiguration -Name "AdminEndpoint" `
    -SecurityDescriptorSddl $sddl
```

Transferência de Arquivos

Cópia Remota

```
# Copiar para servidor remoto
Copy-Item -Path "C:\local\arquivo.txt" `
    -Destination "C:\remoto\" `
    -ToSession $session

# Copiar do servidor remoto
Copy-Item -Path "C:\remoto\logs\" `
    -Destination "C:\local\" `
    -FromSession $session
```

Gerenciamento em Massa

Execução Paralela

```
# Executar em paralelo
$servers = "srv01","srv02","srv03"
$jobs = Invoke-Command -ComputerName $servers `
    -ScriptBlock {
        Get-WindowsUpdate
    } -AsJob

# Aguardar resultados
$results = $jobs | Wait-Job | Receive-Job
```

Fan-out Remoting

```
# Configurar sessões
$sessions = New-PSSession -ComputerName $servers

# Executar comandos
Invoke-Command -Session $sessions -ScriptBlock {
    param($updateType)
    Install-WindowsUpdate -Type $updateType
} -ArgumentList "Security"
```

Troubleshooting

Diagnóstico

```
# Testar conectividade
Test-NetConnection -ComputerName "servidor01" -Port 5985

# Verificar configuração
Get-PSSessionConfiguration
Get-Item WSMAN:\localhost\Client\*

# Logs do WinRM
Get-WinEvent -LogName "Microsoft-Windows-WinRM/Operational"
```

Boas Práticas

Gerenciamento de Sessões

```
# Função helper
function Start-RemoteOperation {
    param(
        [string[]]$ComputerName,
        [scriptblock]$Operation
    )

    try {
        $sessions = New-PSSession -ComputerName $ComputerName
```

```

        Invoke-Command -Session $sessions -ScriptBlock $operation
    }
    catch {
        Write-Error "Falha na operação remota: $_"
    }
    finally {
        $sessions | Remove-PSSession
    }
}

```

Otimização

```

# Minimizar tráfego de rede
$session = New-PSSession -ComputerName "servidor01"
Invoke-Command -Session $session -ScriptBlock {
    $data = Get-Process
    $filtered = $data | Where-Object CPU -gt 50
    return $filtered
}

```

Exercícios Práticos

1. Configuração Básica

```

# Configure remoting em:
# - Servidor local
# - Servidor remoto
# - Autenticação segura

```

2. Gerenciamento Multi-servidor

```

# Desenvolva script para:
# - Monitorar múltiplos servidores

```

```
# - Coletar métricas  
# - Gerar relatório consolidado
```

3. Automação Remota

```
# Crie workflow para:  
# - Atualização de servidores  
# - Backup distribuído  
# - Verificação de saúde
```

Próximos Passos

1. Segurança PowerShell ([Segurança no PowerShell](#))
2. Debugging Remoto ([Debugging no PowerShell](#))
3. DevOps Integration ([PowerShell DevOps Integration](#))

"Com grande poder remoto vem grande responsabilidade."

Segurança no PowerShell

 "Segurança não é uma característica, é uma necessidade."

Políticas de Execução

Configuração de Políticas

```
# Verificar política atual
Get-ExecutionPolicy -List

# Configurar política
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope
CurrentUser
Set-ExecutionPolicy -ExecutionPolicy AllSigned -Scope LocalMachine

# Bypass temporário
powershell.exe -ExecutionPolicy Bypass -File .\script.ps1
```

Assinatura Digital

Certificados e Assinaturas

```
# Criar certificado auto-assinado
$cert = New-SelfSignedCertificate -Subject "CN=PowerShell Code
Signing" `
    -Type CodeSigning -CertStoreLocation "Cert:\CurrentUser\My"

# Assinar script
Set-AuthenticodeSignature -FilePath .\script.ps1 `
    -Certificate $cert
```

JEA (Just Enough Administration)

Configuração JEA

```
# Criar arquivo de configuração
New-PSSessionConfigurationFile -Path .\JEAConfig.pssc `
    -SessionType RestrictedRemoteServer `
    -VisibleCmdlets "Get-Process", "Stop-Process" `
    -VisibleFunctions "Get-CustomInfo"

# Registrar configuração
Register-PSSessionConfiguration -Path .\JEAConfig.pssc `
    -Name "RestrictedSession"
```

Criptografia

Proteção de Dados

```
# Criptografar string
$secureString = ConvertTo-SecureString "SenhaSecreta" `
    -AsPlainText -Force
$encrypted = ConvertFrom-SecureString $secureString

# Descriptografar
$secure = ConvertTo-SecureString $encrypted
$BSTR =
[System.Runtime.InteropServices.Marshal]::SecureStringToBSTR($secure)
$plaintext =
[System.Runtime.InteropServices.Marshal]::PtrToStringAuto($BSTR)
```

HTTPS e SSL/TLS

Configuração de Comunicação Segura

```
# Configurar HTTPS para remoting
winrm quickconfig -transport:https

# Verificar certificados
```

```
Get-ChildItem Cert:\LocalMachine\My
```

```
# Configurar listener HTTPS
```

```
New-WSManInstance winrm/config/Listener `
    -SelectorSet @{Transport="HTTPS"; Address="*"} `
    -ValueSet @{CertificateThumbprint="ABC123..."}
```

Auditoria e Logging

Monitoramento de Atividades

```
# Habilitar logging detalhado
```

```
$logSettings = @{
    LogPipelineExecutionDetails = $true
    LogCommandLifecycleEvents = $true
}
```

```
Set-PSSessionConfiguration -Name Microsoft.PowerShell `
    -PSVersion "7.0" -SecurityDescriptorSddl $sddl @logSettings
```

```
# Verificar logs
```

```
Get-WinEvent -LogName "Windows PowerShell"
```

```
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational"
```

Hardening de Ambiente

Configurações Recomendadas

```
# Desabilitar histórico de comandos
```

```
Set-PSReadLineOption -HistorySaveStyle SaveNothing
```

```
# Limitar módulos carregados
```

```
$env:PSModulePath = "C:\ApprovedModules"
```

```
# Configurar AMSI
```

```
[Net.ServicePointManager]::SecurityProtocol =  
[Net.SecurityProtocolType]::Tls12
```

Boas Práticas

Checklist de Segurança

```
# Função de verificação de segurança  
function Test-SecurityCompliance {  
    $results = @{  
        ExecutionPolicy = Get-ExecutionPolicy  
        PSVersion = $PSVersionTable.PSVersion  
        TLSVersion = [Net.ServicePointManager]::SecurityProtocol  
        AMSIEnabled = Get-MpComputerStatus | Select-Object  
        AMServiceEnabled  
    }  
    return $results  
}
```

Exercícios Práticos

1. Configuração de Ambiente Seguro

```
# Implemente:  
# - Política de execução apropriada  
# - Assinatura de scripts  
# - Logging adequado
```

2. Implementação JEA

```
# Desenvolva:  
# - Configuração JEA personalizada  
# - Roles e capacidades  
# - Testes de segurança
```

Próximos Passos

1. Remoting Seguro ([PowerShell Remoting](#))

2.

3.

"A segurança é como uma cebola: tem várias camadas e pode fazer você chorar."

Debugging no PowerShell

 "Debug é como ser um detetive em um crime que você mesmo cometeu."

Fundamentos de Debugging

Breakpoints

```
# Breakpoint em linha
Set-PSBreakpoint -Script .\script.ps1 -Line 10

# Breakpoint em comando
Set-PSBreakpoint -Command "Get-Process"

# Breakpoint em variável
Set-PSBreakpoint -Variable "errorCount" -Mode Write
```

Debug Interativo

Comandos de Debug

```
# Iniciar sessão de debug
Write-Debug "Iniciando processamento..."
$DebugPreference = "Continue"

# Comandos no debugger
s  # Step into
v  # Step over
o  # Step out
c  # Continue
q  # Quit
k  # Stack trace
```

Logging e Trace

Técnicas de Logging

```
# Verbose output
Write-Verbose "Processando item $i de $total"

# Warning para condições especiais
Write-Warning "Arquivo não encontrado: $file"

# Informação para usuário
Write-Information "Operação completada com sucesso"

# Trace de comandos
Trace-Command -Name ParameterBinding -Expression {
    Get-Process
} -PSHost
```

Ferramentas de Debug

ISE e VSCode

```
# Debug no ISE
F9      # Toggle breakpoint
F5      # Continue
F10     # Step over
F11     # Step into

# Debug no VSCode
# Launch.json configuration
{
    "type": "PowerShell",
    "request": "launch",
    "name": "PowerShell Debug Script",
    "script": "${file}",
    "args": []
}
```

Análise de Performance

Medição e Profiling

```
# Medir tempo de execução
Measure-Command {
    Get-Process | Sort-Object CPU -Descending
}

# Profile de memória
$before = [System.GC]::GetTotalMemory($true)
# ... código a ser analisado ...
$after = [System.GC]::GetTotalMemory($true)
$used = $after - $before
```

Remote Debugging

Debug em Sessões Remotas

```
# Iniciar sessão remota com debug
Enter-PSSession -ComputerName "servidor01" -EnableNetworkAccess

# Debug remoto
Invoke-Command -ComputerName "servidor01" -ScriptBlock {
    Set-PSBreakpoint -Script C:\Scripts\Remote.ps1 -Line 5
}
```

Troubleshooting Comum

Padrões de Debug

```
# Função helper para debug
function Debug-ScriptBlock {
    param(
        [Parameter(Mandatory)]
        [scriptblock]$Code
    )
}
```

```

)

$ErrorActionPreference = 'Stop'
$VerbosePreference = 'Continue'

try {
    & $Code
}
catch {
    Write-Error "Erro: $_"
    Write-Debug "Stack trace: $($_.ScriptStackTrace)"
}
}

```

Boas Práticas

Estratégias de Debug

```

# Estrutura recomendada
function Do-Something {
    [CmdletBinding()]
    param()

    Write-Verbose "Iniciando operação"

    try {
        Write-Debug "Parâmetros validados"
        # código principal
        Write-Verbose "Operação concluída"
    }
    catch {
        Write-Error $_
        throw
    }
}
}

```

Exercícios Práticos

1. Debug Básico

```
# Pratique:  
# - Configurar breakpoints  
# - Usar stepping  
# - Inspecionar variáveis
```

2. Debug Avançado

```
# Implemente:  
# - Debug remoto  
# - Análise de performance  
# - Logging estruturado
```

Próximos Passos

- 1.
- 2.
3. Error Handling ([Tratamento de Erros no PowerShell](#))

"Todo bug é uma oportunidade de aprendizado... ou de questionar suas escolhas de carreira."

Exercícios PowerShell

 "A prática leva à perfeição. Vamos praticar!"

Exercícios Básicos

1. Manipulação de Arquivos

```
# Exercício: Sistema de Backup
# Crie um script que:
# 1. Liste todos arquivos .txt em um diretório
# 2. Copie apenas arquivos modificados nas últimas 24h
# 3. Comprima os arquivos em um zip
# 4. Registre as operações em um log

# Exemplo de solução
$sourceDir = "C:\Dados"
$backupDir = "C:\Backup"
$logFile = "C:\Logs\backup.log"

Get-ChildItem -Path $sourceDir -Filter "*.txt" |
    Where-Object { $_.LastWriteTime -gt (Get-Date).AddDays(-1) } |
    ForEach-Object {
        Copy-Item $_.FullName -Destination $backupDir
        Add-Content $logFile "$(Get-Date) - Copiado: $_.Name"
    }
```

2. Processos e Serviços

```
# Exercício: Monitor de Recursos
# Crie um script que:
# 1. Monitore uso de CPU e memória
# 2. Alerta quando limites forem excedidos
# 3. Registre estatísticas em CSV
# 4. Permita configuração de limites
```

```
# Exemplo de solução
function Monitor-SystemResources {
    param(
        [int]$cpuThreshold = 80,
        [int]$memThreshold = 90
    )

    $stats = Get-Process | Measure-Object -Property CPU, WS -Sum
    $cpuUsage = $stats.Sum / $env:NUMBER_OF_PROCESSORS
    $memUsage = (Get-Counter '\Memory\% Committed Bytes In
Use').CounterSamples.CookedValue

    [PSCustomObject]@{
        DateTime = Get-Date
        CPU = $cpuUsage
        Memory = $memUsage
        Alert = ($cpuUsage -gt $cpuThreshold) -or ($memUsage -gt
$memThreshold)
    }
}
```

Exercícios Intermediários

3. Automação de Rede

```
# Exercício: Diagnóstico de Rede
# Crie um script que:
# 1. Teste conectividade com múltiplos hosts
# 2. Verifique portas específicas
# 3. Meça latência
# 4. Gere relatório HTML

# Exemplo de solução
function Test-NetworkHealth {
    param(
        [string[]]$computers,
```

```

        [int[]]$ports = @(80, 443, 3389)
    )

    $results = foreach ($computer in $computers) {
        $ping = Test-Connection $computer -Count 1 -ErrorAction
        SilentlyContinue
        $portTests = foreach ($port in $ports) {
            $test = Test-NetConnection $computer -Port $port -
            WarningAction SilentlyContinue
            [PSCustomObject]@{
                Port = $port
                Open = $test.TcpTestSucceeded
            }
        }

        [PSCustomObject]@{
            Computer = $computer
            Online = $null -ne $ping
            ResponseTime = $ping.ResponseTime
            Ports = $portTests
        }
    }

    $results | ConvertTo-Html | Out-File "NetworkReport.html"
}

```

4. Gerenciamento de Usuários

```

# Exercício: Sistema de Usuários
# Crie um módulo que:
# 1. Adicione/remova usuários
# 2. Gerencie grupos e permissões
# 3. Gere relatórios de acesso
# 4. Implemente logging

# Exemplo de solução
function New-UserEnvironment {

```

```

param(
    [string]$username,
    [string]$department,
    [string[]]$groups
)

try {
    New-LocalUser -Name $username -Description "Auto created"
    foreach ($group in $groups) {
        Add-LocalGroupMember -Group $group -Member $username
    }

    New-Item -Path "C:\Users\$username\Documents" -ItemType
Directory
    Set-Acl -Path "C:\Users\$username\Documents" -AclObject
$acl

    Write-EventLog -LogName "User Management" -Source
"UserCreation" -EntryType Information -Message "User $username
created successfully"
}
catch {
    Write-Error "Failed to create user environment: $_"
    Write-EventLog -LogName "User Management" -Source
"UserCreation" -EntryType Error -Message "Failed: $_"
}
}

```

Exercícios Avançados

5. Integração com APIs

```

# Exercício: Cliente REST
# Crie um módulo que:
# 1. Interaja com uma API REST
# 2. Implemente autenticação
# 3. Processe respostas JSON

```

4. Trate erros adequadamente

Exemplo de solução

```
function Invoke-ApiRequest {
    param(
        [string]$endpoint,
        [string]$method = "GET",
        [hashtable]$headers,
        [object]$body
    )

    $params = @{
        Uri = $endpoint
        Method = $method
        ContentType = "application/json"
        Headers = $headers
    }

    if ($body) {
        $params.Body = $body | ConvertTo-Json
    }

    try {
        $response = Invoke-RestMethod @params
        return $response
    }
    catch {
        Write-Error "API request failed: $($_.Exception.Message)"
        throw
    }
}
```

6. Projeto Final

```
# Exercício: Sistema de Monitoramento
# Desenvolva um sistema completo que:
# 1. Monitore múltiplos servidores
```

```
# 2. Colete métricas de performance
# 3. Gere alertas e relatórios
# 4. Implemente interface web simples
```

```
# Exemplo de estrutura
```

```
.\MonitoringSystem\
  └─ Modules\
    │   └─ Monitoring.psm1
    │   └─ Reporting.psm1
    │   └─ WebInterface.psm1
  └─ Config\
    │   └─ servers.json
    │   └─ alerts.json
  └─ Scripts\
    │   └─ Start-Monitoring.ps1
    │   └─ Generate-Report.ps1
  └─ Web\
    │   └─ index.html
    │   └─ styles.css
```

Desafios Extra

7. Otimização

```
# Desafio: Otimize um script existente
```

```
# 1. Identifique gargalos
# 2. Implemente parallel processing
# 3. Reduza uso de memória
# 4. Melhore tempo de execução
```

```
# Exemplo de otimização
```

```
$data = 1..1000
```

```
$results = $data | ForEach-Object -ThrottleLimit 10 -Parallel {
    # Processamento pesado aqui
    Start-Sleep -Milliseconds 100
}
```

```
$_  
}
```

8. Segurança

```
# Desafio: Secure Coding  
# 1. Implemente validação de input  
# 2. Use credenciais seguras  
# 3. Aplique princípio do menor privilégio  
# 4. Adicione logging de segurança  
  
# Exemplo de implementação segura  
function Invoke-SecureOperation {  
    [CmdletBinding()]  
    param(  
        [Parameter(Mandatory)]  
        [ValidateNotNullOrEmpty()]  
        [string]$Action,  
  
        [Parameter(Mandatory)]  
        [System.Management.Automation.PSCredential]  
        [System.Management.Automation.Credential()]  
        $Credential  
    )  
  
    try {  
        Start-Transcript -Path ".\SecurityLog.txt" -Append  
  
        $secureAction =  
[System.Web.HttpUtility]::HtmlEncode($Action)  
        Write-Verbose "Executing secure action: $secureAction"  
  
        # Operação principal aqui  
  
        Stop-Transcript  
    }  
    catch {
```

```
        Write-Error "Security violation: $_"  
        throw  
    }  
}
```

Recursos Adicionais

1. PowerShell Gallery (<https://www.powershellgallery.com>)
2. Pester Testing Framework (<https://pester.dev>)
3. PSScriptAnalyzer (<https://github.com/PowerShell/PSScriptAnalyzer>)

Próximos Passos

1. Módulos Avançados ([Módulos PowerShell](#))
2. DevOps Integration ([PowerShell DevOps Integration](#))
3. Segurança Avançada ([Segurança no PowerShell](#))

"A melhor maneira de aprender é fazendo. A segunda melhor é quebrando algo e tendo que consertar."

PowerShell Integration

 "Integrando PowerShell com as principais plataformas e serviços cloud."

Visão Geral

O PowerShell é uma ferramenta poderosa para integração com diversas plataformas cloud e DevOps:

- Azure Integration ([Azure PowerShell Integration](#))
- AWS Integration ([AWS PowerShell Integration](#))
- DevOps Integration ([PowerShell DevOps Integration](#))

Conceitos Fundamentais

Autenticação e Segurança

```
# Gerenciamento seguro de credenciais
$credentials = Get-Credential
$secureString = ConvertTo-SecureString "token" -AsPlainText -Force

# Armazenamento seguro
$encrypted = $secureString | ConvertFrom-SecureString
Set-Content ".\credentials.txt" $encrypted
```

REST APIs

```
# Função base para chamadas API
function Invoke-APICall {
    param(
        [string]$Uri,
        [string]$Method = "GET",
        [hashtable]$Headers,
```

```
        $Body
    )

    $params = @{
        Uri = $Uri
        Method = $Method
        ContentType = "application/json"
        Headers = $Headers
    }

    if ($Body) {
        $params.Body = ($Body | ConvertTo-Json)
    }

    Invoke-RestMethod @params
}
```

Boas Práticas

1. Segurança

- Use gerenciamento seguro de credenciais
- Implemente retry com backoff
- Valide certificados SSL/TLS
- Monitore e registre acessos

2. Performance

- Utilize parallel processing
- Implemente caching quando apropriado
- Otimize chamadas de API
- Gerencie recursos adequadamente

3. Manutenibilidade

- Modularize código
- Documente integrações
- Implemente logging
- Use controle de versão

Padrões Comuns

Retry Pattern

```
function Invoke-WithRetry {
    param(
        [scriptblock]$ScriptBlock,
        [int]$MaxAttempts = 3,
        [int]$DelaySeconds = 5
    )

    $attempt = 1
    $success = $false

    do {
        try {
            & $ScriptBlock
            $success = $true
            break
        }
        catch {
            Write-Warning "Tentativa $attempt de $MaxAttempts
falhou"

            if ($attempt -lt $MaxAttempts) {
                Start-Sleep -Seconds ($DelaySeconds * $attempt)
                $attempt++
            }
            else {
```

```

        throw
    }
}
} while ($attempt -le $MaxAttempts)
}

```

Circuit Breaker

```

class CircuitBreaker {
    [int]$FailureThreshold = 3
    [int]$ResetTimeoutSeconds = 60
    [datetime]$LastFailureTime
    [int]$FailureCount = 0
    [bool]$IsOpen = $false

    [bool] CanProcess() {
        if (-not $this.IsOpen) { return $true }

        if ((Get-Date) - $this.LastFailureTime).TotalSeconds -gt
        $this.ResetTimeoutSeconds) {
            $this.Reset()
            return $true
        }

        return $false
    }

    [void] RecordFailure() {
        $this.FailureCount++
        $this.LastFailureTime = Get-Date

        if ($this.FailureCount -ge $this.FailureThreshold) {
            $this.IsOpen = $true
        }
    }

    [void] Reset() {

```

```
        $this.FailureCount = 0
        $this.IsOpen = $false
    }
}
```

Ferramentas Recomendadas

IDEs e Editores

- Visual Studio Code + PowerShell Extension
- PowerShell ISE
- Azure Data Studio

Ferramentas de Teste

- Pester
- PSScriptAnalyzer
- Azure PowerShell Test Framework

Monitoramento

- Azure Monitor
- AWS CloudWatch
- Application Insights

Próximos Passos

1. Azure Integration ([Azure PowerShell Integration](#))

- Azure PowerShell
- Azure CLI

- Azure REST APIs

2. AWS Integration ([AWS PowerShell Integration](#))

- AWS Tools for PowerShell
- AWS CLI
- AWS SDKs

3. DevOps Integration ([PowerShell DevOps Integration](#))

- CI/CD Pipelines
- Infrastructure as Code
- Automação

Recursos Adicionais

- Azure PowerShell Docs (<https://docs.microsoft.com/powershell/azure>)
- AWS Tools for PowerShell (<https://aws.amazon.com/powershell>)
- DevOps with PowerShell (<https://docs.microsoft.com/azure/devops>)

"Integração é a arte de fazer diferentes sistemas trabalharem em harmonia."

Azure PowerShell Integration

 "Automatize e gerencie recursos Azure com PowerShell."

Configuração Inicial

Instalação

```
# Instalar Azure PowerShell
Install-Module -Name Az -AllowClobber -Scope CurrentUser

# Verificar instalação
Get-InstalledModule -Name Az -AllVersions
```

Autenticação

```
# Login interativo
Connect-AzAccount

# Login não-interativo
$credential = Get-Credential
Connect-AzAccount -Credential $credential

# Service Principal
$sp = @{
    SubscriptionId = "your-sub-id"
    TenantId = "your-tenant-id"
    ApplicationId = "your-app-id"
    CertificateThumbprint = "cert-thumbprint"
}
Connect-AzAccount -ServicePrincipal @sp
```

Operações Comuns

Resource Groups

```
# Criar Resource Group
New-AzResourceGroup -Name "MyRG" -Location "eastus2"

# Listar Resource Groups
Get-AzResourceGroup | Select-Object ResourceGroupName, Location

# Remover Resource Group
Remove-AzResourceGroup -Name "MyRG" -Force
```

Virtual Machines

```
# Criar VM
$vmParams = @{
    ResourceGroupName = "MyRG"
    Name = "MyVM"
    Location = "eastus2"
    Size = "Standard_DS2_v2"
    Image = "Win2019Datacenter"
}
New-AzVM @vmParams

# Gerenciar estado
Stop-AzVM -ResourceGroupName "MyRG" -Name "MyVM"
Start-AzVM -ResourceGroupName "MyRG" -Name "MyVM"
```

Storage

```
# Criar Storage Account
$storageParams = @{
    ResourceGroupName = "MyRG"
    Name = "mystorageaccount"
    Location = "eastus2"
    SkuName = "Standard_LRS"
}
New-AzStorageAccount @storageParams
```

```
# Upload de blob
$context = (Get-AzStorageAccount -ResourceGroupName "MyRG" -Name
"mystorageaccount").Context
Set-AzStorageBlobContent -Container "mycontainer" -File
"myfile.txt" -Context $context
```

Automação e DevOps

ARM Templates

```
# Deploy ARM template
$templateParams = @{
    ResourceGroupName = "MyRG"
    TemplateFile = "template.json"
    TemplateParameterFile = "parameters.json"
}
New-AzResourceGroupDeployment @templateParams
```

Azure Functions

```
# Deploy Function App
$functionParams = @{
    ResourceGroupName = "MyRG"
    Name = "MyFunctionApp"
    Runtime = "PowerShell"
    StorageAccountName = "mystorageaccount"
}
New-AzFunctionApp @functionParams
```

Monitoramento

Azure Monitor

```
# Obter métricas
Get-AzMetric -ResourceId $resourceId -MetricName "CPU" -TimeGrain
```

```
00:01:00
```

```
# Configurar alertas
$alertParams = @{
    ResourceGroupName = "MyRG"
    Name = "HighCPULAlert"
    Location = "eastus2"
    TargetResourceId = $resourceId
    MetricName = "Percentage CPU"
    Operator = "GreaterThan"
    Threshold = 90
}
New-AzMetricAlertRule @alertParams
```

Boas Práticas

1. Gerenciamento de Credenciais

```
# Usar Key Vault
$secret = Get-AzKeyVaultSecret -VaultName "MyVault" -Name
"MySecret"
$credential = New-Object PSCredential "user", $secret.SecretValue
```

2. Error Handling

```
try {
    $result = New-AzResourceGroup -Name "MyRG" -Location "eastus2"
    Write-Output "Resource Group criado:
$(($result.ResourceGroupName)"
}
catch {
    Write-Error "Falha ao criar Resource Group: $_"
    throw
}
finally {
```

```
    Disconnect-AzAccount  
}
```

3. Logging

```
# Configurar logging  
Enable-AzContextAutosave  
Start-Transcript -Path ".\AzureDeployment.log"  
  
# Executar operações  
# ...  
  
Stop-Transcript
```

Scripts de Exemplo

1. Backup Automatizado

```
function Backup-AzureVM {  
    param(  
        [string]$ResourceGroupName,  
        [string]$VMName  
    )  
  
    # Criar snapshot do disco  
    $vm = Get-AzVM -ResourceGroupName $ResourceGroupName -Name  
$VMName  
    $disk = Get-AzDisk -ResourceGroupName $ResourceGroupName -  
DiskName $vm.StorageProfile.OsDisk.Name  
  
    $snapshotConfig = New-AzSnapshotConfig -SourceUri $disk.Id -  
Location $disk.Location -CreateOption Copy  
    $snapshot = New-AzSnapshot -ResourceGroupName  
$ResourceGroupName -SnapshotName "$VMName-snap-$(Get-Date -Format  
'yyyyMMdd') " -Snapshot $snapshotConfig
```

```
    return $snapshot  
}
```

2. Deployment Automatizado

```
function Deploy-AzureEnvironment {  
    param(  
        [string]$Environment,  
        [string]$Location  
    )  
  
    # Criar infraestrutura base  
    $rgName = "RG-$Environment"  
    New-AzResourceGroup -Name $rgName -Location $Location  
  
    # Deploy recursos  
    $resources = @(  
        @{ Type = "Storage"; Name = "storage$Environment" }  
        @{ Type = "AppService"; Name = "app$Environment" }  
        @{ Type = "SQL"; Name = "sql$Environment" }  
    )  
  
    foreach ($resource in $resources) {  
        Write-Output "Deploying $($resource.Type):  
$($resource.Name)"  
        # Lógica de deployment específica para cada tipo  
    }  
}
```

Próximos Passos

1. AWS Integration ([AWS PowerShell Integration](#))
2. DevOps Integration ([PowerShell DevOps Integration](#))
3. Security Best Practices ([Segurança no PowerShell](#))

Recursos Adicionais

- Azure PowerShell Documentation (<https://docs.microsoft.com/powershell/azure/>)
- Azure Resource Manager Templates (<https://docs.microsoft.com/azure/azure-resource-manager/templates/>)
- Azure DevOps (<https://docs.microsoft.com/azure/devops/>)

"Azure e PowerShell: a combinação perfeita para automação cloud."

AWS PowerShell Integration

 "Gerencie sua infraestrutura AWS com o poder do PowerShell."

Configuração Inicial

Instalação

```
# Instalar AWS Tools
Install-Module -Name AWS.Tools.Installer
Install-AWSToolsModule AWS.Tools.EC2, AWS.Tools.S3,
AWS.Tools.Common

# Verificar instalação
Get-AWSPowerShellVersion
```

Autenticação

```
# Configurar credenciais
Set-AWSCredential -AccessKey "YOUR-ACCESS-KEY" -SecretKey "YOUR-
SECRET-KEY" -StoreAs "default"

# Usar perfil
Initialize-AWSDefaultConfiguration -ProfileName "default" -Region
"us-east-1"
```

Serviços Principais

EC2 (Elastic Compute Cloud)

```
# Listar instâncias
Get-EC2Instance

# Criar instância
$launchParams = @{
```

```

    ImageId = "ami-12345678"
    InstanceType = "t2.micro"
    KeyName = "my-key-pair"
    SecurityGroupId = "sg-12345678"
}
New-EC2Instance @launchParams

# Gerenciar estado
Stop-EC2Instance -InstanceId "i-1234567890abcdef0"
Start-EC2Instance -InstanceId "i-1234567890abcdef0"

```

S3 (Simple Storage Service)

```

# Criar bucket
New-S3Bucket -BucketName "my-unique-bucket"

# Upload de arquivo
Write-S3Object -BucketName "my-bucket" -File "local-file.txt" -Key
"remote-file.txt"

# Download de arquivo
Read-S3Object -BucketName "my-bucket" -Key "remote-file.txt" -File
"local-copy.txt"

```

Lambda

```

# Criar função
$functionCode = @'
exports.handler = async (event) => {
    return {
        statusCode: 200,
        body: JSON.stringify('Hello from Lambda!')
    };
};
'@

```

```

$function = New-AWSFunction `
    -FunctionName "MyFunction" `
    -Runtime "nodejs14.x" `
    -Role "arn:aws:iam::123456789012:role/lambda-role" `
    -Code_ZipFile
([System.Text.Encoding]::UTF8.GetBytes($functionCode))

# Invocar função
Invoke-LMFunction -FunctionName "MyFunction"

```

Automação e DevOps

CloudFormation

```

# Deploy stack
New-CFNStack `
    -StackName "MyStack" `
    -TemplateBody (Get-Content "template.json" -Raw) `
    -Parameter @(
        @{ ParameterKey="Environment"; ParameterValue="Production"
    }
    )

# Verificar status
Get-CFNStack -StackName "MyStack"

```

Systems Manager

```

# Executar comando
$command = Send-SSMCommand `
    -InstanceId "i-1234567890abcdef0" `
    -DocumentName "AWS-RunShellScript" `
    -Parameter @{
        commands = @("echo Hello World")
    }

```

```
# Obter resultado
Get-SSMCommandInvocation `
    -CommandId $command.CommandId `
    -Details $true
```

Monitoramento

CloudWatch

```
# Obter métricas
$metric = Get-CWMetricStatistic `
    -Namespace "AWS/EC2" `
    -MetricName "CPUUtilization" `
    -Dimension @{Name="InstanceId";Value="i-1234567890abcdef0"} `
    -StartTime (Get-Date).AddHours(-1) `
    -EndTime (Get-Date) `
    -Period 300 `
    -Statistic "Average"

# Criar alarme
Write-CWMetricAlarm `
    -AlarmName "HighCPU" `
    -MetricName "CPUUtilization" `
    -Namespace "AWS/EC2" `
    -Statistic "Average" `
    -Period 300 `
    -Threshold 90 `
    -ComparisonOperator "GreaterThanThreshold" `
    -EvaluationPeriods 2
```

Scripts de Exemplo

1. Backup Automatizado EC2

```
function Backup-EC2Instance {
    param(
```

```

        [string]$InstanceId,
        [string]$Description
    )

    try {
        # Criar snapshot de todos os volumes
        $volumes = Get-EC2Volume |
            Where-Object { $_.Attachments.InstanceId -eq
$InstanceId }

        foreach ($volume in $volumes) {
            $snapshot = New-EC2Snapshot `
                -VolumeId $volume.VolumeId `
                -Description "$Description-$(Get-Date -Format
'yyyyMMdd')"

            Write-Output "Created snapshot:
$($snapshot.SnapshotId)"
        }
    }
    catch {
        Write-Error "Backup failed: $_"
        throw
    }
}

```

2. Deployment Multi-Region

```

function Deploy-MultiRegion {
    param(
        [string[]]$Regions,
        [string]$StackName,
        [string]$TemplatePath
    )

    foreach ($region in $Regions) {
        Set-DefaultAWSRegion -Region $region
    }
}

```

```

    Write-Output "Deploying to $region..."
    New-CFNStack `
        -StackName "$StackName-$region" `
        -TemplateBody (Get-Content $TemplatePath -Raw) `
        -Parameter @(
            @{ ParameterKey="Region"; ParameterValue=$region }
        )
    }
}

```

Boas Práticas

1. Segurança

```

# Usar roles e políticas IAM
# Evitar hardcoding de credenciais
# Implementar princípio do menor privilégio

```

2. Error Handling

```

try {
    $result = New-EC2Instance @launchParams
}
catch [Amazon.EC2.AmazonEC2Exception] {
    Write-Error "EC2 error: $_"
    throw
}
catch {
    Write-Error "Unexpected error: $_"
    throw
}

```

3. Logging

```
# Configurar logging AWS
Set-AWSLogging -LogMetrics -LogResponses

# Logging local
Start-Transcript -Path ".\AWSDeployment.log"
```

Próximos Passos

1. Azure Integration ([Azure PowerShell Integration](#))
2. DevOps Integration ([PowerShell DevOps Integration](#))
3. Security Best Practices ([Segurança no PowerShell](#))

Recursos Adicionais

- AWS Tools for PowerShell Documentation (<https://docs.aws.amazon.com/powershell/>)
- AWS CloudFormation (<https://aws.amazon.com/cloudformation/>)
- [AWS Systems Manager](https://aws.amazon

PowerShell DevOps Integration

 "Automatize seu pipeline DevOps com o poder do PowerShell."

CI/CD Integration

Azure DevOps

```
# Configurar Azure DevOps CLI
az extension add --name azure-devops

# Criar pipeline
$pipelineYaml = @'
trigger:
- main

pool:
  vmImage: 'windows-latest'

steps:
- task: PowerShell@2
  inputs:
    targetType: 'inline'
    script: |
      Write-Host "Executando pipeline..."
'@

az pipelines create --name "MyPipeline" --yaml-path /azure-
pipelines.yml
```

Jenkins

```
# Script para Jenkins pipeline
function Invoke-JenkinsPipeline {
    param(
```

```

        [string]$JenkinsUrl,
        [string]$JobName,
        [PSCredential]$Credential
    )

    $auth = [Convert]::ToBase64String(
        [Text.Encoding]::ASCII.GetBytes(
            "$($Credential.UserName):$($Credential.GetNetworkCredential().Password)"
        )
    )

    $headers = @{
        Authorization = "Basic $auth"
    }

    Invoke-RestMethod -Uri "$JenkinsUrl/job/$JobName/build" -
    Headers $headers -Method Post
}

```

Infrastructure as Code

Terraform Integration

```

function Deploy-TerraformInfra {
    param(
        [string]$WorkingDirectory,
        [string]$Environment
    )

    Push-Location $WorkingDirectory

    try {
        # Inicializar Terraform
        terraform init
    }
}

```

```

    # Selecionar workspace
    terraform workspace select $Environment

    # Planejar mudanças
    terraform plan -out=tfplan

    # Aplicar mudanças
    terraform apply tfplan
}
finally {
    Pop-Location
}
}

```

ARM Templates

```

function Deploy-ArmTemplate {
    param(
        [string]$ResourceGroupName,
        [string]$TemplateFile,
        [hashtable]$Parameters
    )

    New-AzResourceGroupDeployment `
        -ResourceGroupName $ResourceGroupName `
        -TemplateFile $TemplateFile `
        -TemplateParameterObject $Parameters `
        -Mode Incremental
}

```

Container Orchestration

Docker Integration

```

function Build-DockerImage {
    param(

```

```

        [string]$DockerfilePath,
        [string]$ImageName,
        [string]$Tag = "latest"
    )

    docker build -t "${ImageName}:${Tag}" -f $DockerfilePath .

    if ($LASTEXITCODE -eq 0) {
        Write-Output "Image built successfully:
${ImageName}:${Tag}"
    }
    else {
        throw "Docker build failed"
    }
}

```

Kubernetes Management

```

function Deploy-KubernetesApp {
    param(
        [string]$Namespace,
        [string]$DeploymentFile
    )

    # Verificar conexão com cluster
    kubectl cluster-info
    if ($LASTEXITCODE -ne 0) {
        throw "Kubernetes cluster não acessível"
    }

    # Criar/atualizar namespace
    kubectl create namespace $Namespace --dry-run=client -o yaml |
    kubectl apply -f -

    # Aplicar deployment

```

```
kubectl apply -f $DeploymentFile -n $Namespace  
}
```

Monitoramento e Logging

Prometheus Integration

```
function Get-PrometheusMetrics {  
    param(  
        [string]$PrometheusUrl,  
        [string]$Query  
    )  
  
    $endpoint = "$PrometheusUrl/api/v1/query"  
    $response = Invoke-RestMethod -Uri "$endpoint?query=$Query"  
  
    return $response.data.result  
}
```

Grafana Dashboard

```
function New-GrafanaDashboard {  
    param(  
        [string]$GrafanaUrl,  
        [string]$ApiKey,  
        [string]$DashboardJson  
    )  
  
    $headers = @{  
        Authorization = "Bearer $ApiKey"  
        'Content-Type' = 'application/json'  
    }  
  
    Invoke-RestMethod `br/>        -Uri "$GrafanaUrl/api/dashboards/db" `br/>        -Method Post `
```

```
-Headers $headers `
-Body $DashboardJson
}
```

Boas Práticas

1. Versionamento

- Use controle de versão para scripts
- Mantenha documentação atualizada
- Implemente testes automatizados
- Siga convenções de nomenclatura

2. Segurança

- Utilize secrets management
- Implemente RBAC
- Audite execuções
- Valide inputs

3. Automação

- Crie pipelines reutilizáveis
- Automatize testes
- Implemente validações
- Configure monitoramento

Próximos Passos

1. Azure Integration ([Azure PowerShell Integration](#))
2. AWS Integration ([AWS PowerShell Integration](#))
3. Security Best Practices ([Segurança no PowerShell](#))

Recursos Adicionais

- Azure DevOps Documentation (<https://docs.microsoft.com/azure/devops>)
- Jenkins Documentation (<https://www.jenkins.io/doc>)
- Terraform Documentation (<https://www.terraform.io/docs>)
- Kubernetes Documentation (<https://kubernetes.io/docs>)

"DevOps é sobre cultura, automação é apenas uma ferramenta para alcançá-la."

Comparação de Terminais

 "Escolher o terminal certo é como escolher a ferramenta certa para o trabalho."

Visão Geral

A escolha do terminal correto pode impactar significativamente sua produtividade. Cada terminal tem seus pontos fortes e fracos:

Terminais Windows

- Command Prompt (CMD)
- PowerShell
- Windows Terminal
- ConEmu/Cmder

Terminais Unix-like

- Bash
- Zsh
- Fish
- Terminal.app (macOS)

Critérios de Comparação

Performance

- Velocidade de execução
- Consumo de memória
- Eficiência em scripts

Funcionalidades

- Suporte a scripts
- Integração com sistema
- Extensibilidade
- Customização

Usabilidade

- Curva de aprendizado
- Interface
- Documentação
- Comunidade

Guias Detalhados

1. CMD vs PowerShell ([CMD vs PowerShell: Duelo de Terminais](#))
2. Windows vs Unix ([Windows vs Unix Terminals](#))
3. Alternativas de Terminal ([Alternativas de Terminal](#))

Matriz de Decisão

Cenário	Terminal Recomendado	Razão
Scripts legados	CMD	Compatibilidade
Automação Windows	PowerShell	Recursos nativos
Dev Web	WSL/Bash	Compatibilidade Unix
DevOps	PowerShell	Integração cloud

"O melhor terminal é aquele que resolve seu problema atual."

CMD vs PowerShell: Duelo de Terminais

⚠ "Tradição encontra modernidade no mundo dos terminais Windows."

Comparação Rápida

Aspecto	CMD	PowerShell
Lançamento	1981 (MS-DOS)	2006
Base	Texto	Objetos .NET
Scripting	Batch (.bat/.cmd)	PowerShell (.ps1)
Case Sensitivity	Não	Sim (parcial)
Pipeline	Texto	Objetos

Pontos Fortes

CMD

```
:: Vantagens CMD
- Simplicidade
- Baixo consumo
- Compatibilidade legada
- Scripts batch simples
- Presente em todo Windows
```

PowerShell

```
# Vantagens PowerShell
- Orientado a objetos
- Mais poderoso
- Integração .NET
- Remoting nativo
- Módulos extensíveis
```

Casos de Uso

Quando Usar CMD

- Scripts legados
- Tarefas simples
- Sistemas limitados
- Compatibilidade máxima
- Performance crítica

Quando Usar PowerShell

- Automação complexa
- Administração Windows
- DevOps/Cloud
- Scripting avançado
- Integração sistemas

Exemplos Comparativos

Listagem de Processos

```
:: CMD
tasklist
```

```
# PowerShell
Get-Process | Select-Object Name, CPU, Memory
```

Manipulação de Arquivos

```
:: CMD
dir /s /b *.txt > files.txt

# PowerShell
Get-ChildItem -Recurse -Filter *.txt | Select-Object FullName |
Export-Csv files.csv
```

Migração CMD para PowerShell

Equivalências Comuns

CMD	PowerShell
dir	Get-ChildItem
copy	Copy-Item
del	Remove-Item
type	Get-Content

"CMD é como um canivete: simples e confiável. PowerShell é como um canivete suíço: versátil e poderoso."

Windows vs Unix Terminals

⚠ "Dois mundos, duas filosofias, infinitas possibilidades."

Diferenças Fundamentais

Filosofia

- Windows: Integração sistema e GUI
- Unix: "Tudo é arquivo", composição

Estrutura

Aspecto	Windows	Unix
Separador	\	/
Case	Insensitive	Sensitive
Extensões	Importantes	Opcionais
EOL	CRLF	LF

Comparação de Comandos

Navegação

```
# Windows (CMD/PowerShell)
dir
cd
path

# Unix
```

```
ls
cd
$PATH
```

Manipulação Arquivos

```
# Windows
copy
move
del

# Unix
cp
mv
rm
```

Ambiente de Desenvolvimento

Windows

```
# Vantagens
- Integração Visual Studio
- .NET nativo
- PowerShell avançado
- WSL disponível

# Desvantagens
- Menos ferramentas dev
- Configuração complexa
- Performance I/O menor
```

Unix

```
# Vantagens
- Ferramentas nativas
- Package managers
```

- Performance I/O
- Containers nativos

Desvantagens

- GUI limitada
- Fragmentação distros
- Curva aprendizado

Soluções Híbridas

WSL (Windows Subsystem for Linux)

- # Melhor dos dois mundos
- Linux nativo no Windows
- Acesso sistemas arquivo
- Docker integrado
- VS Code remote

Cygwin/MSYS2

- Emulação Unix no Windows
- Compatibilidade parcial
- Performance menor
- Útil legado

Escolhendo seu Ambiente

Desenvolvimento Web

- **Recomendado:** WSL/Unix
- **Razão:** Ferramentas nativas

Enterprise/.NET

- **Recomendado:** Windows
- **Razão:** Integração ecosystem

DevOps

- **Recomendado:** Híbrido
- **Razão:** Flexibilidade máxima

"Não é sobre qual é melhor, é sobre qual é melhor para você."

Alternativas de Terminal

 "Para cada desenvolvedor, um terminal perfeito."

Windows Terminal

Características

```
# Vantagens
- Múltiplas tabs
- Temas e customização
- GPU acceleration
- Unicode/emoji
- Split panes
```

Configuração

```
{
  "defaultProfile": "{PowerShell-GUID}",
  "profiles": {
    "list": [
      {
        "name": "PowerShell",
        "colorScheme": "One Half Dark",
        "fontFace": "CascadiaCode NF"
      }
    ]
  }
}
```

ConEmu/Cmder

Recursos

- Tabs e splits
- Quake-style dropdown
- Integração Git
- Portable
- Tasks customizáveis

Cenários Uso

- Desenvolvimento Windows
- Múltiplos terminais
- Customização avançada
- Sistemas legados

Hyper

Features

```
// Vantagens
- Cross-platform
- Extensível (JavaScript)
- Temas comunidade
- Plugin system
- Web technologies
```

Plugins Essenciais

- hyper-material-theme
- hyper-search
- hyper-tabs-enhanced
- hyper-statusline

Alacritty

Características

- GPU-accelerated
- Configuração YAML
- Minimalista
- Performance máxima
- Cross-platform

Performance

```
# Configuração otimizada
env:
  TERM: xterm-256color
window:
  dimensions:
    columns: 120
    lines: 30
  dynamic_padding: true
```

Tabela Comparativa

Terminal	Performance	Customização	Recursos	Uso Memória
Windows Terminal	Alta	Média	Alto	Média
ConEmu	Média	Alta	Alto	Média
Hyper	Baixa	Alta	Médio	Alta
Alacritty	Muito Alta	Baixa	Baixo	Baixa

Escolhendo seu Terminal

Desenvolvimento Web

- **Recomendado:** Windows Terminal + WSL
- **Alternativa:** Hyper

Enterprise

- **Recomendado:** ConEmu
- **Alternativa:** Windows Terminal

Performance

- **Recomendado:** Alacritty
- **Alternativa:** Windows Terminal

Customização

- **Recomendado:** Hyper
- **Alternativa:** ConEmu

Dicas de Customização

Fontes

- Cascadia Code
- FiraCode
- JetBrains Mono
- Hack Nerd Font

Color Schemes

- Dracula
- Nord
- Solarized
- One Dark

Produtividade

- Oh My Posh
- Terminal Icons
- PSReadLine
- Git Integration

Próximos Passos

1. CMD Basics ([CMD: Seu Primeiro Terminal](#))
2. PowerShell Essentials ([Introdução ao PowerShell](#))
3. Best Practices ([Melhores Práticas](#))

"O terminal perfeito é aquele que você esquece que está usando."

Referência Completa



"Uma referência sólida é a base do conhecimento prático."

Estrutura da Referência

Documentação Principal

- Referência CMD ([Referência CMD](#))
- Referência PowerShell ([Referência PowerShell](#))
- Glossário ([Glossário](#))

Guias Práticos

- Melhores Práticas ([Melhores Práticas](#))
- Troubleshooting ([Troubleshooting Guide](#))
- Cheat Sheet ([Terminal Cheat Sheet](#))

Como Usar esta Referência

Busca Rápida

1. Use o índice de comandos
2. Consulte exemplos práticos
3. Verifique casos comuns
4. Explore soluções alternativas

Aprendizado Estruturado

1. Comece pelo básico

2. Pratique exemplos
3. Explore avançado
4. Customize soluções

Mantenha-se Atualizado

- Versões Windows
- Atualizações PowerShell
- Novas features
- Security patches

"Uma boa referência é como um mapa: mostra todos os caminhos possíveis."

Referência CMD

 "Todo comando tem seu propósito, todo parâmetro sua razão."

Comandos Essenciais

Navegação

```
cd [path]          :: Muda diretório
dir [options]       :: Lista arquivos
path [options]      :: Mostra/define PATH
```

Arquivos

```
copy [src] [dst]   :: Copia arquivos
move [src] [dst]   :: Move arquivos
del [file]          :: Deleta arquivos
ren [old] [new]     :: Renomeia
```

Sistema

```
tasklist           :: Lista processos
taskkill           :: Termina processo
systeminfo         :: Info sistema
sfc                :: System File Checker
```

Parâmetros Comuns

Diretórios

Parâmetro	Descrição
/s	Recursivo
/a	Atributos
/b	Básico
/p	Pausa

Arquivos

Parâmetro	Uso
/y	Confirma
/v	Verbose
/q	Quiet
/f	Força

Operadores

Redirecionamento

```
>   :: Saída
>> :: Append
<   :: Entrada
|   :: Pipe
```

Condicionais

```
&&  :: AND
||   :: OR
^    :: Escape
```

Variáveis Ambiente

Sistema

```
%PATH%      :: Caminhos
%SYSTEMROOT% :: Windows
%TEMP%       :: Temporário
%USERPROFILE% :: Usuário
```

Customizadas

```
set VAR=valor  :: Define
echo %VAR%     :: Usa
setx VAR valor  :: Permanente
```

Códigos de Retorno

Código	Significado
0	Sucesso
1	Erro geral
2	Arquivo não encontrado
3	Path não encontrado

Exemplos Práticos

Backup

```
:: Backup com data
for /f "tokens=1-3 delims=/ " %%a in ('%date%') do (
    set BACKUP=backup_%%c%%a%%b
)
xcopy /s /i "source" "%BACKUP%"
```

Limpeza

```
:: Limpa temporários
del /s /q %temp%\*.*
for /d %%x in (%temp%\*) do rd /s /q "%%x"
```

Troubleshooting

Erros Comuns

1. 'X' não é reconhecido

- Verificar PATH
- Confirmar extensão

2. Acesso negado

- Executar como admin
- Verificar permissões

Dicas Avançadas

Performance

```
:: Desativa echo
@echo off
:: Processa rápido
setlocal enabledelayedexpansion
```

Debugging

```
:: Debug mode  
echo on  
set  
pause
```

"Um comando bem documentado vale por mil tentativas e erros."

Referência PowerShell

 "PowerShell: onde comandos encontram objetos."

Cmdlets Essenciais

Navegação

```
# Sistema de arquivos
Get-Location      # pwd
Set-Location      # cd
Get-ChildItem     # ls/dir
```

Manipulação

```
# Arquivos e objetos
Get-Content       # cat
Set-Content       # >
Copy-Item         # cp
Move-Item         # mv
Remove-Item       # rm
```

Pipeline

```
# Operações
Select-Object     # select
Where-Object      # filter
ForEach-Object    # loop
Sort-Object       # sort
Group-Object      # group
```

Tipos de Dados

Básicos

```
# Tipos comuns
[string]      # Texto
[int]         # Inteiro
[bool]        # Boolean
[datetime]    # Data/hora
[array]       # Array
```

Complexos

```
# Estruturas
[hashtable]   # Dictionary
[PSCustomObject] # Objeto
[xml]         # XML
[scriptblock] # Code block
```

Operadores

Comparação

```
-eq  # Igual
-ne  # Diferente
-gt  # Maior
-lt  # Menor
-like # Pattern
-match # Regex
```

Lógicos

```
-and # AND
-or  # OR
-not # NOT
-xor # XOR
```

Fluxo de Controle

Condicionais

```
if ($condition) {  
    # code  
}  
elseif ($other) {  
    # code  
}  
else {  
    # code  
}  
  
switch ($value) {  
    condition { code }  
    default { code }  
}
```

Loops

```
foreach ($item in $collection) {}  
while ($condition) {}  
do {} while ($condition)  
for ($i=0; $i -lt 10; $i++) {}
```

Error Handling

Try-Catch

```
try {  
    # código arriscado  
}  
catch [Exception] {  
    # tratamento  
}  
finally {
```

```
# cleanup  
}
```

Preferências

```
$ErrorActionPreference = 'Stop'  
$WarningPreference = 'SilentlyContinue'  
$VerbosePreference = 'Continue'
```

Módulos

Gestão

```
Get-Module  
Import-Module  
Install-Module  
Update-Module
```

Comuns

- PSReadLine
- Az
- AWS.Tools
- PSScriptAnalyzer

Perfis

Localizações

```
$PROFILE.CurrentUserCurrentHost  
$PROFILE.CurrentUserAllHosts  
$PROFILE.AllUsersCurrentHost  
$PROFILE.AllUsersAllHosts
```

Segurança

Execution Policy

```
Get-ExecutionPolicy  
Set-ExecutionPolicy RemoteSigned
```

Assinatura

```
Set-AuthenticodeSignature  
Get-AuthenticodeSignature
```

"Com grande poder vem grande responsabilidade de documentar."

Glossário

 "Entender os termos é o primeiro passo para dominar a tecnologia."

A

Alias

Comando alternativo/atalho para outro comando.

Ambiente (Environment)

Conjunto de variáveis e configurações do sistema.

API

Interface de Programação de Aplicações.

B

Batch

Script ou arquivo de comandos (.bat/.cmd).

Binary

Arquivo executável ou programa compilado.

C

CLI

Interface de Linha de Comando.

Cmdlet

Comando nativo do PowerShell.

D

Directory

Pasta ou diretório no sistema de arquivos.

Domain

Domínio de rede Windows.

E

Environment Variable

Variável de ambiente do sistema.

Execution Policy

Política de execução de scripts.

F

File System

Sistema de arquivos.

Function

Bloco de código reutilizável.

G

GUI

Interface Gráfica do Usuário.

GUID

Identificador Único Global.

H

Hash

Valor único gerado de dados.

Host

Sistema ou computador na rede.

I

IDE

Ambiente de Desenvolvimento Integrado.

Instance

Instância de programa ou serviço.

J

Job

Tarefa em background.

JSON

JavaScript Object Notation.

K

Kernel

Núcleo do sistema operacional.

Key

Chave (registro, criptografia).

L

Library

Biblioteca de código.

Log

Registro de eventos.

M

Module

Módulo PowerShell.

MSI

Instalador Windows.

N

Namespace

Espaço de nomes.

NuGet

Gerenciador de pacotes .NET.

O

Object

Objeto (PowerShell/programação).

Output

Saída de comando.

P

Parameter

Parâmetro de comando.

Pipeline

Encadeamento de comandos.

Q

Query

Consulta (SQL, WMI).

Queue

Fila de processos.

R

Registry

Registro do Windows.

Remote

Acesso/operação remota.

S

Script

Arquivo com comandos.

Shell

Interface de comando.

T

Task

Tarefa agendada.

Thread

Linha de execução.

U

UNC

Convenção de Nomenclatura Universal.

URI

Identificador de Recurso Universal.

V

Variable

Variável (ambiente/script).

VBScript

Visual Basic Script.

W

WMI

Windows Management Instrumentation.

Workflow

Fluxo de trabalho automatizado.

X

XML

eXtensible Markup Language.

XPath

XML Path Language.

Y

YAML

YAML Ain't Markup Language.

Z

ZIP

Arquivo compactado.

Zone

Zona de segurança.

"Conhecimento começa com vocabulário."

Melhores Práticas

 "Boas práticas hoje, menos problemas amanhã."

Organização

Estrutura de Scripts

```
# Estrutura recomendada
|- scripts/
  |- modules/
  |- functions/
  |- configs/
  |- logs/
  |- tests/
```

Nomenclatura

```
# Convenções
Verb-Noun      # Cmdlets
PascalCase     # Funções
camelCase      # Variáveis
snake_case     # Arquivos
```

Código

Documentação

```
# Comentários úteis
<#
.SYNOPSIS
    Breve descrição
.DESCRIPTION
    Descrição detalhada
.PARAMETER Param
```

```
    Descrição do parâmetro
.EXAMPLE
    Exemplo de uso
#>
```

Tratamento de Erros

```
# Error handling
try {
  # Validar inputs
  # Usar try-catch
  # Logging adequado
  # Cleanup resources
}
catch {
  # Mensagens claras
  # Logging estruturado
  # Fallback gracioso
}
```

Segurança

Princípios

1. Menor privilégio
2. Validação inputs
3. Sanitização dados
4. Logs seguros

Práticas

```
# Segurança básica
- Use SSL/TLS
- Encrypt secrets
```

- Audit access
- Update regular

Performance

Otimização

- # Dicas performance
- Cache results
- Batch operations
- Parallel when possible
- Monitor resources

Recursos

- # Gestão recursos
- Close connections
- Dispose objects
- Clear variables
- Garbage collect

Manutenção

Versionamento

- # Controle versão
- Use git
- Semantic versions
- Change logs
- Tags releases

Testing

- # Testes
- Unit tests
- Integration

- Regression
- Performance

Automação

CI/CD

```
# Pipeline
stages:
  - lint
  - test
  - build
  - deploy
```

Monitoring

```
# Monitoramento
- Logs centrais
- Métricas
- Alertas
- Dashboard
```

Colaboração

Code Review

1. Legibilidade
2. Segurança
3. Performance
4. Standards

Knowledge Share

1. Documentação
2. Wiki
3. Treinamento
4. Mentoria

Checklist

Desenvolvimento

- ☐ Documentado
- ☐ Testado
- ☐ Revisado
- ☐ Otimizado

Deployment

- ☐ Backup
- ☐ Rollback
- ☐ Monitoring
- ☐ Support

"Qualidade não é acidente, é resultado de boas práticas."

Troubleshooting Guide



"Problemas são oportunidades de aprendizado disfarçadas."

Problemas Comuns

Acesso Negado

```
# Sintomas
- "Access Denied"
- Error 5
- UnauthorizedAccess

# Soluções
1. Verificar permissões
2. Executar como admin
3. Checar ACLs
4. Validar tokens
```

Comando Não Encontrado

```
# Sintomas
- "'X' is not recognized"
- "Command not found"

# Soluções
1. Verificar PATH
2. Confirmar instalação
3. Checar extensão
4. Validar ambiente
```

Diagnóstico

Logs

```
# Locais importantes
- Event Viewer
- System logs
- App logs
- Security logs
```

Ferramentas

```
# Diagnóstico
- Process Monitor
- Network Monitor
- Resource Monitor
- Performance Monitor
```

Network Issues

Conectividade

```
# Testes básicos
Test-NetConnection
ping
tracert
nslookup
```

Portas

```
# Verificação
netstat -ano
Test-NetConnection -Port
Get-NetTCPConnection
```

Performance

CPU

```
# Análise CPU
Get-Process | Sort CPU -Descending
typeperf "\Processor(_Total)\% Processor Time"
```

Memória

```
# Memória
Get-Process | Sort WS -Descending
Get-Counter '\Memory\Available MBytes'
```

Segurança

Permissões

```
# Verificação
Get-Acl
icacls
whoami /all
```

Certificados

```
# Certificados
Get-ChildItem Cert:\
certutil
```

Scripts

Debugging

```
# Técnicas
Set-PSBreakpoint
Write-Debug
$DebugPreference = 'Continue'
```

Logging

```
# Logging estruturado
Start-Transcript
Write-EventLog
Add-Content log.txt
```

Recovery

Backup

```
# Backup
- System Restore
- File History
- Shadow Copy
- Export settings
```

Restore

```
# Restore
- Safe Mode
- Last Known Good
- Recovery Console
- System Reset
```

Checklist

Básico

1. Verificar erros óbvios
2. Consultar logs
3. Testar reprodução
4. Documentar passos

Avançado

1. Análise root cause
2. Implementar fix
3. Testar solução
4. Prevenir recorrência

Recursos

Documentação

- Microsoft Docs
- PowerShell Docs
- TechNet
- Stack Overflow

Ferramentas

- Sysinternals
- PowerShell ISE
- VSCode
- Process Explorer

"O melhor troubleshooting é aquele que previne o problema."

Terminal Cheat Sheet

 "Seu guia rápido para comandos essenciais."

CMD Essencial

Navegação

```
cd [path]      # Muda diretório
dir             # Lista arquivos
cd ..          # Volta diretório
cd \           # Raiz do drive
```

Arquivos

```
copy           # Cópia
move           # Move
del            # Deleta
ren            # Renomeia
```

Sistema

```
tasklist       # Lista processos
taskkill       # Mata processo
systeminfo     # Info sistema
ipconfig       # Config rede
```

PowerShell Básico